dp_murankova_pouzivatelska_prirucka

April 14, 2025

1 DP - Metódy hlbokého učenia pre podporu spracovania dát z meteorických kamier AMOS

1.1 Používateľská príručka

1.1.1 Balíky

[ ]:
```
!pip install deap fvcore hdbscan numpy opencv-python pandas plotly scikit-learn␣
↪scikit-optimize transformers thop timm torch torchvision ultralytics
```

1.1.2 Trénovanie modelov

Vlastná CNN

[ ]:
```
import os
import time
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
import matplotlib.pyplot as plt
from PIL import Image

# Cesty k dátam
data_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
↪DP_rozdelenie_dat/data_split_v2/"
train_dir = os.path.join(data_dir, "train", "images")
valid_dir = os.path.join(data_dir, "valid", "images")
test_dir = os.path.join(data_dir, "test", "images")

train_label_dir = os.path.join(data_dir, "train", "labels")
valid_label_dir = os.path.join(data_dir, "valid", "labels")
test_label_dir = os.path.join(data_dir, "test", "labels")

# Predspracovanie dát
transform = transforms.Compose([
```

```
    transforms.Resize((480, 480)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# Dataset
class MeteorDataset(Dataset):
    def __init__(self, image_dir, label_dir, transform=None):
        self.image_dir = image_dir
        self.label_dir = label_dir
        self.image_paths = [os.path.join(image_dir, fname) for fname in os.
 ↪listdir(image_dir) if fname.endswith('.jpg')]
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image_path = self.image_paths[idx]
        label_path = os.path.join(self.label_dir, os.path.basename(image_path).
 ↪replace('.jpg', '.txt'))

        image = Image.open(image_path).convert('RGB')

        if os.path.exists(label_path) and open(label_path).read().strip() ==␣
 ↪"0":
            label = 1  # Meteor prítomný
        else:
            label = 0  # Meteor neprítomný

        if self.transform:
            image = self.transform(image)

        return image, label

# Načítanie dát
train_dataset = MeteorDataset(train_dir, train_label_dir, transform)
valid_dataset = MeteorDataset(valid_dir, valid_label_dir, transform)

train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=4, shuffle=False)

# Definícia CNN modelu
class CNNModel(nn.Module):
```

```
    def __init__(self):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.dropout = nn.Dropout(0.3)

        # Výpočet veľkosti FC vrstvy
        sample_input = torch.randn(1, 3, 480, 480)
        with torch.no_grad():
            feature_size = self._get_feature_map_size(sample_input)

        self.fc1 = nn.Linear(feature_size, 128)
        self.fc2 = nn.Linear(128, 1)
        self.sigmoid = nn.Sigmoid()

    def _get_feature_map_size(self, x):
        x = torch.relu(self.conv1(x))
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv2(x))
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv3(x))
        x = torch.max_pool2d(x, 2)
        return x.view(1, -1).size(1)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv2(x))
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv3(x))
        x = torch.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = self.dropout(x)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        x = self.sigmoid(x)
        return x

# Inicializácia
model = CNNModel().cuda()
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.000001, weight_decay=1e-4)

# Tréning
num_epochs = 50
train_losses = []
```

```
valid_losses = []
train_accuracies = []
valid_accuracies = []

# Early stopping
early_stopping_patience = 5
best_valid_loss = float('inf')
epochs_without_improvement = 0

for epoch in range(num_epochs):
    start_time = time.time()

    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for inputs, labels in train_loader:
        inputs, labels = inputs.cuda(), labels.cuda().float()

        optimizer.zero_grad()
        outputs = model(inputs).squeeze(1)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        predicted = (outputs > 0.5).float()
        correct += (predicted == labels).sum().item()
        total += labels.size(0)

    train_losses.append(running_loss / len(train_loader))
    train_accuracies.append(correct / total)

    # Validácia
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, labels in valid_loader:
            inputs, labels = inputs.cuda(), labels.cuda().float()
            outputs = model(inputs).squeeze(1)
            loss = criterion(outputs, labels)

            running_loss += loss.item()
            predicted = (outputs > 0.5).float()
```

```
            correct += (predicted == labels).sum().item()
            total += labels.size(0)

    valid_loss = running_loss / len(valid_loader)
    valid_acc = correct / total

    valid_losses.append(valid_loss)
    valid_accuracies.append(valid_acc)

    end_time = time.time()
    epoch_duration = end_time - start_time

    print(f"Epoch {epoch+1}/{num_epochs} - "
          f"Train Loss: {train_losses[-1]:.4f}, Train Accuracy:␣
↪{train_accuracies[-1]*100:.2f}% - "
          f"Valid Loss: {valid_loss:.4f}, Valid Accuracy: {valid_acc*100:.2f}%␣
↪- "
          f"Epoch Time: {epoch_duration:.2f}s")

    # Early stopping kontrola
    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        epochs_without_improvement = 0
        torch.save(model.state_dict(), 'best_model_early_stopping.pth')
    else:
        epochs_without_improvement += 1
        if epochs_without_improvement >= early_stopping_patience:
            print(f"Early stopping triggered after {epoch+1} epochs.")
            break

# Grafy
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.plot(train_accuracies, label='Train Accuracy')
plt.plot(valid_accuracies, label='Valid Accuracy')
plt.title('Accuracy during Training')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(train_losses, label='Train Loss')
plt.plot(valid_losses, label='Valid Loss')
plt.title('Loss during Training')
plt.xlabel('Epochs')
plt.ylabel('Loss')
```

```
plt.legend()

plt.tight_layout()
plt.show()

# Uloženie finálneho modelu
torch.save(model.state_dict(), 'meteor_detection_cnn_model_v4.pth')
```

ConvNext-Small

[ ]:
```
import os
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import timm
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler
from torchvision import transforms
from PIL import Image
from tqdm import tqdm
from transformers import get_cosine_schedule_with_warmup

# Dataset class
class MeteorDataset(Dataset):
    def __init__(self, image_dir, label_dir, transform=None):
        self.image_dir = image_dir
        self.label_dir = label_dir
        self.transform = transform
        self.image_names = [fname for fname in os.listdir(image_dir) if fname.
 ↪endswith(('.jpg', '.png'))]
        self.image_paths = [os.path.join(image_dir, fname) for fname in self.
 ↪image_names]
        self.label_paths = [os.path.join(label_dir, fname.replace('.jpg', '.
 ↪txt').replace('.png', '.txt')) for fname in self.image_names]

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]
        label_path = self.label_paths[idx]
        img = Image.open(img_path).convert("RGB")

        if os.path.exists(label_path):
            with open(label_path, 'r') as f:
                content = f.read().strip()
                label = 1 if '0' in content else 0
```

```
        else:
            label = 0

        if self.transform:
            img = self.transform(img)
        return img, torch.tensor(label, dtype=torch.float32)

# Augmentations
train_transforms = transforms.Compose([
    #transforms.RandomResizedCrop(720, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1),
    transforms.ToTensor(),
    #transforms.RandomErasing(p=0.2, scale=(0.02, 0.1), ratio=(0.3, 3.3)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

valid_transforms = transforms.Compose([
    transforms.Resize((720, 720)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# Datasets
data_path = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪DP_rozdelenie_dat/data_split_v2"
train_dataset = MeteorDataset(f"{data_path}/train/images", f"{data_path}/train/
 ↪labels", transform=train_transforms)
valid_dataset = MeteorDataset(f"{data_path}/valid/images", f"{data_path}/valid/
 ↪labels", transform=valid_transforms)

# Weighted sampler
label_list = [int(train_dataset.__getitem__(idx)[1].item()) for idx in␣
 ↪range(len(train_dataset))]
num_non_meteor, num_meteor = label_list.count(0), label_list.count(1)
print(f"  Meteors: {num_meteor}, Non-Meteors: {num_non_meteor}")
weights = [1.0 / num_non_meteor if label == 0 else 1.0 / num_meteor for label␣
 ↪in label_list]
sampler = WeightedRandomSampler(weights, num_samples=len(label_list),␣
 ↪replacement=True)

batch_size = 8
train_loader = DataLoader(train_dataset, batch_size=batch_size, sampler=sampler)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)
```

```python
# Focal Loss
class FocalLoss(nn.Module):
    def __init__(self, alpha=0.75, gamma=2):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.bce = nn.BCEWithLogitsLoss(reduction='none')

    def forward(self, inputs, targets):
        targets = targets.unsqueeze(1)
        BCE_loss = self.bce(inputs, targets)
        pt = torch.exp(-BCE_loss)
        F_loss = self.alpha * (1 - pt) ** self.gamma * BCE_loss
        return F_loss.mean()

# Model setup
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = timm.create_model("convnext_small", pretrained=True, num_classes=1).
 ↪to(device)
model.classifier = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(model.num_features, 1)
)

# Training setup
criterion = FocalLoss(alpha=0.75, gamma=2)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, weight_decay=2e-4)

epochs = 30
num_training_steps = len(train_loader) * epochs
num_warmup_steps = len(train_loader) * 5  # 5 warmup epoch steps
scheduler = get_cosine_schedule_with_warmup(optimizer, num_warmup_steps,␣
 ↪num_training_steps)

# Training loop
best_val_loss = float('inf')
patience, trigger_times = 5, 0

for epoch in range(epochs):
    model.train()
    total_loss = 0
    for images, labels in tqdm(train_loader, desc=f"Epoch {epoch+1}/{epochs}"):
        images, labels = images.to(device), labels.to(device).float()
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
```

```
        optimizer.step()
        scheduler.step()
        total_loss += loss.item()

    avg_train_loss = total_loss / len(train_loader)

    # Validation
    model.eval()
    val_loss = 0
    with torch.no_grad():
        for images, labels in valid_loader:
            images, labels = images.to(device), labels.to(device).float()
            outputs = model(images)
            val_loss += criterion(outputs, labels).item()

    avg_val_loss = val_loss / len(valid_loader)

    print(f"Epoch {epoch+1}: Train Loss={avg_train_loss:.4f}, Val␣
↪Loss={avg_val_loss:.4f}")

    # Early stopping
    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        torch.save(model.state_dict(), "datasetv2_best_convnext_model_v65.pth")
        print("  Model saved (best so far)")
        trigger_times = 0
    else:
        trigger_times += 1
        print(f"  Early stopping counter: {trigger_times}/{patience}")
        if trigger_times >= patience:
            print("  Early stopping triggered")
            break

print("  Training completed!")
```

EfficientNet-B4

[ ]:
```
import os
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import timm
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler
from torchvision import transforms
from PIL import Image
from tqdm import tqdm
```

```
import matplotlib.pyplot as plt

class MeteorDataset(Dataset):
    def __init__(self, image_dir, label_dir, transform=None):
        self.image_dir = image_dir
        self.label_dir = label_dir
        self.transform = transform
        self.image_names = [fname for fname in os.listdir(image_dir) if fname.
 ↪endswith('.jpg') or fname.endswith('.png')]
        self.image_paths = [os.path.join(image_dir, fname) for fname in self.
 ↪image_names]
        self.label_paths = [os.path.join(label_dir, fname.replace('.jpg', '.
 ↪txt').replace('.png', '.txt')) for fname in self.image_names]

        # Check that label files exist for every image and count statistics
        self.meteor_count = 0
        self.non_meteor_count = 0
        for label_path in self.label_paths:
            if not os.path.exists(label_path):
                raise FileNotFoundError(f"Label file {label_path} does not␣
 ↪exist!")

            with open(label_path, 'r') as f:
                content = f.read().strip()

            if content == '0':
                self.meteor_count += 1  # Meteor
            else:
                self.non_meteor_count += 1  # Non-meteor (empty label)

        total = len(self.label_paths)
        print(f"  Dataset loaded: {total} samples (Meteors: {self.
 ↪meteor_count}, Non-meteors: {self.non_meteor_count})")
        print(f"  Class ratio - Meteors: {self.meteor_count / total:.2%},␣
 ↪Non-meteors: {self.non_meteor_count / total:.2%}")

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]
        label_path = self.label_paths[idx]

        # Load image
        img = Image.open(img_path).convert("RGB")
```

```
        # Load label
        with open(label_path, 'r') as f:
            content = f.read().strip()

        label = 1 if content == '0' else 0  # 1 = Meteor, 0 = Non-meteor

        # Apply transform if any
        if self.transform:
            img = self.transform(img)

        return img, torch.tensor(label, dtype=torch.float32)

# Stronger Augmentations
transform = transforms.Compose([
    transforms.Resize((720, 720)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1),
    transforms.ToTensor(),
    #transforms.RandomErasing(p=0.2, scale=(0.02, 0.1), ratio=(0.3, 3.3)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# Load datasets
batch_size = 8
train_dataset = MeteorDataset(
    "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪DP_rozdelenie_dat/data_split_v2/train/images",
    "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪DP_rozdelenie_dat/data_split_v2/train/labels",
    transform
)

valid_dataset = MeteorDataset(
    "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪DP_rozdelenie_dat/data_split_v2/valid/images",
    "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪DP_rozdelenie_dat/data_split_v2/valid/labels",
    transform
)

# Compute Class Weights
num_non_meteor = sum(1 for _, label in train_dataset if label == 0)
num_meteor = len(train_dataset) - num_non_meteor
weights = [num_non_meteor / len(train_dataset) if label == 1 else num_meteor /␣
 ↪len(train_dataset) for _, label in train_dataset]
```

```
sampler = WeightedRandomSampler(weights, len(weights), replacement=True)

train_loader = DataLoader(train_dataset, batch_size=batch_size, sampler=sampler)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)

# Focal Loss
class FocalLoss(nn.Module):
    def __init__(self, alpha=1, gamma=2, logits=True):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.logits = logits

    def forward(self, inputs, targets):
        BCE_loss = nn.functional.binary_cross_entropy_with_logits(inputs,␣
 ↪targets, reduction='none') if self.logits else nn.functional.
 ↪binary_cross_entropy(inputs, targets, reduction='none')
        pt = torch.exp(-BCE_loss)
        return (self.alpha * (1 - pt) ** self.gamma * BCE_loss).mean()

# Model, Optimizer, Scheduler
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = timm.create_model("efficientnet_b4", pretrained=True, num_classes=1).
 ↪to(device)
criterion = FocalLoss()
optimizer = optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min',␣
 ↪factor=0.5, patience=3, verbose=True)

# Training Loop
epochs = 30
best_val_loss = float("inf")
early_stop_patience = 5
early_stop_counter = 0
train_losses, val_losses, val_accuracies = [], [], []

for epoch in range(epochs):
    model.train()
    total_train_loss = 0.0
    for images, labels in tqdm(train_loader, desc=f"Epoch {epoch+1}/{epochs}␣
 ↪[Training]"):
        images, labels = images.to(device), labels.to(device).unsqueeze(1)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

```
        total_train_loss += loss.item()

    avg_train_loss = total_train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    model.eval()
    total_val_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in tqdm(valid_loader, desc=f"Epoch {epoch+1}/
↪{epochs} [Validation]"):
            images, labels = images.to(device), labels.to(device).unsqueeze(1)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_val_loss += loss.item()
            preds = (torch.sigmoid(outputs) > 0.5).int()
            correct += (preds == labels.int()).sum().item()
            total += labels.size(0)

    avg_val_loss = total_val_loss / len(valid_loader)
    val_losses.append(avg_val_loss)
    val_accuracy = 100 * correct / total
    val_accuracies.append(val_accuracy)

    print(f" Epoch {epoch+1}: Train Loss={avg_train_loss:.4f}, Val␣
↪Loss={avg_val_loss:.4f}, Val Acc={val_accuracy:.2f}%")
    scheduler.step(avg_val_loss)

    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        torch.save(model.state_dict(), "best_efficientNetB4_model_v6.pth")
        print(" Model saved!")
        early_stop_counter = 0
    else:
        early_stop_counter += 1
        print(f" Early stop counter: {early_stop_counter}/
↪{early_stop_patience}")

    if early_stop_counter >= early_stop_patience:
        print(" Early stopping!")
        break

# Plotting results
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.legend()
```

```
plt.title('Loss')

plt.subplot(1, 2, 2)
plt.plot(val_accuracies, label='Validation Accuracy', color='green')
plt.legend()
plt.title('Accuracy')
plt.show()

# Load best model
def test_model(model, test_loader):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for images, labels in tqdm(test_loader):
            images, labels = images.to(device), labels.to(device).unsqueeze(1)
            outputs = model(images)
            preds = (torch.sigmoid(outputs) > 0.5).int()
            correct += (preds == labels.int()).sum().item()
            total += labels.size(0)
    print(f"  Test Accuracy: {100 * correct / total:.2f}%")

model.load_state_dict(torch.load("best_efficientNetB4_model_v6.pth"))
model.to(device)
```

1.1.3 Technické parametre

[ ]:
```
import torch
import torch.nn as nn
import timm

from ultralytics import YOLO
from thop import profile
```

Yolo

[ ]:
```
model_path = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
  ↪najlepsie_modely/najlepsie_modely/yolov12_vacsie_obrazky.pt"
model = YOLO(model_path)

#print(model)
#print(model.model)
print(model.model.__class__)
```

[ ]:
```
# yolov12
```

```
model = torch.load("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov12_vacsie_obrazky.
 ↪pt", map_location=torch.device("cpu"))
print(model.keys())
print(model['train_args'])
print(model['docs'])
```

[ ]:
```
model = YOLO("/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/najlepsie_modely/yolov12_vacsie_obrazky.pt")

model.eval()

# Dummy input of size 1408 x 1408
dummy_input = torch.randn(1, 3, 1408, 1408)

flops, params = profile(model.model, inputs=(dummy_input,), verbose=False)

print(f"FLOPs: {flops / 1e9:.2f} GFLOPs")
print(f"Parametre: {params / 1e6:.2f} miliónov")
```

[ ]:
```
model = torch.load("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov11_vacsie_obrazky.
 ↪pt", map_location=torch.device("cpu"))

def count_conv_layers(model):
    conv_count = sum(1 for layer in model.modules() if isinstance(layer, nn.
 ↪Conv2d))
    return conv_count

conv_layers_count = count_conv_layers(model['model'])
print(conv_layers_count)
```

ConvNext-Small

[ ]:
```
model = timm.create_model("convnext_small", pretrained=True, num_classes=1)

model_path = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/najlepsie_modely/datasetv2_best_convnext_model_v5.pth"
state_dict = torch.load(model_path, map_location=torch.device("cpu"))

# get rid of bias
state_dict.pop("classifier.bias", None)

# load weights
model.load_state_dict(state_dict, strict=False)

def count_conv_layers(model):
```

```
    return sum(1 for layer in model.modules() if isinstance(layer, torch.nn.
 ↪Conv2d))

conv_layers_count = count_conv_layers(model)
print(f"Počet konvolučných vrstiev: {conv_layers_count}")
```

[ ]:
```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

# ConvNeXt-Small
model_convnext = timm.create_model("convnext_small", pretrained=True,␣
 ↪num_classes=1)
print(f"ConvNeXt-Small: {count_parameters(model_convnext) / 1e6:.2f}M␣
 ↪parametrov")
```

[ ]:
```
convnext_small = timm.create_model("convnext_small", pretrained=False)

# Input tensor of size (1, 3, 720, 720)
input_tensor = torch.randn(1, 3, 720, 720)

# Count FLOPs and params
flops_convnext, params_convnext = profile(convnext_small,␣
 ↪inputs=(input_tensor,))

print(f"ConvNeXt-Small: {flops_convnext / 1e9:.2f} GFLOPs, {params_convnext /␣
 ↪1e6:.2f}M parametrov")
```

EfficientNet-B4

[ ]:
```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

# EfficientNet-B4
model_effnet = timm.create_model("efficientnet_b4", pretrained=True,␣
 ↪num_classes=1)
print(f"EfficientNet-B4: {count_parameters(model_effnet) / 1e6:.2f}M␣
 ↪parametrov")
```

[ ]:
```
efficientnet_b4 = timm.create_model("efficientnet_b4", pretrained=False)

# Input tensor of size (1, 3, 720, 720)
input_tensor = torch.randn(1, 3, 720, 720)

# Count FLOPs and params
flops_efficientnet, params_efficientnet = profile(efficientnet_b4,␣
 ↪inputs=(input_tensor,))
```

```
print(f"EfficientNet-B4: {flops_efficientnet / 1e9:.2f} GFLOPs,␣
 ↪{params_efficientnet / 1e6:.2f}M parametrov")
```

Vlastná CNN

[ ]:
```
# SE Block
class SEBlock(nn.Module):
    def __init__(self, in_channels, reduction=16):
        super(SEBlock, self).__init__()
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(in_channels, in_channels // reduction),
            nn.ReLU(),
            nn.Linear(in_channels // reduction, in_channels),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        y = self.pool(x).view(b, c)
        y = self.fc(y).view(b, c, 1, 1)
        return x * y

# CNN model w SE blocks
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.se1 = SEBlock(32)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.se2 = SEBlock(64)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
        self.se3 = SEBlock(128)
        self.dropout = nn.Dropout(0.3)

        # count input size - conv + pool
        sample_input = torch.randn(1, 3, 320, 480)
        with torch.no_grad():
            feature_size = self._get_feature_map_size(sample_input)

        self.fc1 = nn.Linear(feature_size, 128)
        self.fc2 = nn.Linear(128, 1)

    def _get_feature_map_size(self, x):
        x = torch.relu(self.conv1(x))
        x = self.se1(x)
        x = torch.max_pool2d(x, 2)
```

```
        x = torch.relu(self.conv2(x))
        x = self.se2(x)
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv3(x))
        x = self.se3(x)
        x = torch.max_pool2d(x, 2)
        return x.view(1, -1).size(1)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.se1(x)
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv2(x))
        x = self.se2(x)
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv3(x))
        x = self.se3(x)
        x = torch.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = self.dropout(x)
        x = torch.relu(self.fc1(x))
        return torch.sigmoid(self.fc2(x))

model = CNNModel()
model.eval()

dummy_input = torch.randn(1, 3, 480, 480)

flops, params = profile(model, inputs=(dummy_input,), verbose=False)

print(f"FLOPs: {flops / 1e9:.2f} GFLOPs")
print(f"Parametre: {params / 1e6:.2f} miliónov")
```

1.1.4 Vyhodnotenie

[2]:
```
import os
import pandas as pd
import numpy as np

import torch
import torch.nn as nn
import timm
from torchvision import transforms
from tqdm import tqdm
from ultralytics import YOLO

import plotly.graph_objects as go
```

```
import plotly.subplots as sp
from PIL import Image
from sklearn.metrics import confusion_matrix, precision_score, recall_score,␣
 ↪f1_score, accuracy_score, precision_recall_curve
```

[3]:
```
# Calculating the overlap coefficient (overlap of distributions)
def compute_overlap(pos_scores, neg_scores, bins=20):
    pos_hist, bin_edges = np.histogram(pos_scores, bins=bins, range=(0, 1),␣
 ↪density=True)
    neg_hist, _ = np.histogram(neg_scores, bins=bins, range=(0, 1),␣
 ↪density=True)
    overlap = np.sum(np.minimum(pos_hist, neg_hist)) * (bin_edges[1] -␣
 ↪bin_edges[0])
    return overlap * 100

# Evaluation and visualization
def evaluate_and_plot_plotly(df, nazov, threshold=0.5):
    # Colors
    royal_blue = '#4169E1'
    yellow = '#FFD700'
    light_gray = '#F5F5F5'

    y_true, y_scores = df['true_label'].values, df['confidence'].values

    # Precision-Recall curve + F1
    precision_vals, recall_vals, thresholds = precision_recall_curve(y_true,␣
 ↪y_scores)
    f1_scores = 2 * (precision_vals * recall_vals) / (precision_vals +␣
 ↪recall_vals + 1e-6)

    max_f1_idx = np.argmax(f1_scores)
    best_f1_precision = precision_vals[max_f1_idx]
    best_f1_recall = recall_vals[max_f1_idx]

    max_precision_idx = np.argmax(precision_vals)
    best_prec_precision = precision_vals[max_precision_idx]
    best_prec_recall = recall_vals[max_precision_idx]

    # Confussion Matrix
    y_pred = (y_scores >= threshold).astype(int)
    cm = confusion_matrix(y_true, y_pred, labels=[1, 0])
    TP, FN = cm[0]
    FP, TN = cm[1]

    precision = precision_score(y_true, y_pred, zero_division=0)
    recall = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
```

```
    accuracy = accuracy_score(y_true, y_pred)

    # Overlap coefficient (OVL)
    pos_scores = y_scores[y_true == 1]
    neg_scores = y_scores[y_true == 0]
    overlap_percent = compute_overlap(pos_scores, neg_scores)

    # Confussion Matrix
    cm_text = np.array([
        [f"TP = {TP}", f"FN = {FN}"],
        [f"FP = {FP}", f"TN = {TN}"]
    ])

    cm_reordered = np.array([
        [TP, FN],
        [FP, TN]
    ])

    # Subplots
    fig = sp.make_subplots(
        rows=1, cols=3,
        subplot_titles=(
            f"Kontingenčná tabuľka (Prah {threshold:.2f})",
            "PR Krivka",
            f"Distribúcia Pravdepodobností<br>Prekrytie: {overlap_percent:.2f}%"
        )
    )

    # Confussion Matrix
    heatmap = go.Heatmap(
        z=cm_reordered,
        colorscale=[[0, light_gray], [1, royal_blue]],
        showscale=False,
        text=cm_text,
        texttemplate="%{text}"
    )

    fig.add_trace(heatmap, row=1, col=1)

    # Add Descriptions to Confussion Matrix
    fig.add_annotation(
        x=-1.1, y=0.5, text="Skutočnosť",
        showarrow=False, font=dict(size=20), textangle=-90,
        xanchor="center", yanchor="middle", row=1, col=1
    )

    fig.add_annotation(
```

```
        x=0.5, y=2, text="Predikcia",
        showarrow=False, font=dict(size=20),
        xanchor="center", row=1, col=1
    )

    fig.add_annotation(
        x=0, y=1.7, text="S meteorom",
        showarrow=False, font=dict(size=16),
        xanchor="center", row=1, col=1
    )

    fig.add_annotation(
        x=1, y=1.7, text="Bez meteoru",
        showarrow=False, font=dict(size=16),
        xanchor="center", row=1, col=1
    )

    fig.add_annotation(
        x=-0.8, y=0, text="Bez meteoru",
        showarrow=False, font=dict(size=16), textangle=-90,
        yanchor="middle", xanchor="center", row=1, col=1
    )

    fig.add_annotation(
        x=-0.8, y=1, text="S meteorom",
        showarrow=False, font=dict(size=16), textangle=-90,
        yanchor="middle", xanchor="center", row=1, col=1
    )

    # PR Curve
    fig.add_trace(
        go.Scatter(
            x=recall_vals, y=precision_vals,
            mode='lines', line=dict(color=royal_blue, width=3),
            name='PR Krivka'
        ),
        row=1, col=2
    )

    # Max F1 Point
    fig.add_trace(
        go.Scatter(
            x=[best_f1_recall], y=[best_f1_precision],
            mode='markers+text', marker=dict(color=yellow, size=16,␣
↪symbol='circle'),
            text=["Max F1 Skóre"], textposition="bottom left",
            name='Max F1 Skóre'
```

```
        ),
        row=1, col=2
    )

    # Max Precision Point
    fig.add_trace(
        go.Scatter(
            x=[best_prec_recall],
            y=[best_prec_precision],
            mode='markers+text',
            marker=dict(color=yellow, size=16, symbol='diamond'),
            text=["Max Presnosť"],
            textposition="bottom right",
            name='Max Presnosť'
        ),
        row=1, col=2
    )

    # Point for the current decision threshold on the PR curve
    threshold_idx = np.argmin(np.abs(thresholds - threshold))
    fig.add_trace(
        go.Scatter(
            x=[recall_vals[threshold_idx]],
            y=[precision_vals[threshold_idx]],
            mode='markers+text',
            marker=dict(color='black', size=16, symbol='circle'),
            text=[f"Prah {threshold:.2f}"], textposition="top left",
            name='Prah'
        ),
        row=1, col=2
    )

    # Probability histogram
    fig.add_trace(
        go.Histogram(
            x=pos_scores, name='S meteorom',
            marker_color=royal_blue, opacity=0.75, nbinsx=20, showlegend=True
        ),
        row=1, col=3
    )

    fig.add_trace(
        go.Histogram(
            x=neg_scores, name='Bez meteoru',
            marker_color=yellow, opacity=0.75, nbinsx=20, showlegend=True
        ),
        row=1, col=3
```

```
    )

    # Vertical line for the current threshold in the histogram
    fig.add_shape(
        type="line",
        x0=threshold, x1=threshold,
        y0=0, y1=1,
        line=dict(color="black", width=2, dash="dash"),
        xref='x3',
        yref='paper'  # relative to the chart height
    )

    # Layout and Design
    fig.update_layout(
        title=dict(
            text="Vyhodnotenie modelu: " + nazov,
            x=0.5, xanchor='center',
            y=0.98,
            font=dict(size=24)
        ),
        width=1600, height=520,
        barmode='overlay',
        plot_bgcolor=light_gray,
        paper_bgcolor='white',
        font=dict(size=16, color='black'),
        margin=dict(l=100, r=2, t=120, b=20),
        legend=dict(orientation="h", yanchor="bottom", y=1.13,␣
↪xanchor="center", x=0.5, font=dict(size=16))
    )

    for annotation in fig['layout']['annotations']:
        annotation['font'] = dict(size=20)

    fig.update_xaxes(showgrid=False)
    fig.update_yaxes(showgrid=False)

    # Descriptions of axes
    fig.update_xaxes(title_text="Návratnosť (Recall)", row=1, col=2)
    fig.update_yaxes(title_text="Presnosť (Precision)", row=1, col=2)
    fig.update_xaxes(title_text="Pravdepodobnosť meteoru", row=1, col=3)
    fig.update_yaxes(title_text="Počet obrázkov", row=1, col=3)

    fig.update_xaxes(zeroline=False, row=1, col=1, showticklabels=False)
    fig.update_yaxes(zeroline=False, row=1, col=1, showticklabels=False)

    fig.update_xaxes(zeroline=False, row=1, col=2, showticklabels=True)  # keep␣
↪the axis for PR curve
```

```
    fig.update_yaxes(zeroline=False, row=1, col=2, showticklabels=True)

    fig.update_xaxes(zeroline=False, row=1, col=3, showticklabels=True)  # keep␣
 ↪the axis for histogram
    fig.update_yaxes(zeroline=False, row=1, col=3, showticklabels=True)

    # Results for the current threshold
    print(f"\n Výsledky pre prah {threshold:.2f}:")
    print(f"  TP (Správne meteory): {TP}")
    print(f"  TN (Správne nemeteory): {TN}")
    print(f"  FP (Falošné meteory): {FP}")
    print(f"  FN (Nezachytené meteory): {FN}")
    print(f"  Presnosť (Precision): {precision:.4f}")
    print(f"  Návratnosť (Recall): {recall:.4f}")
    print(f"  F1-Skóre: {f1:.4f}")
    print(f"  Presnosť klasifikácie (Accuracy): {accuracy:.4f}")
    print(f"  Prekrytie distribúcie (Overlap coefficient): {overlap_percent:.
 ↪2f}%")

    fig.show()

    # Optimized points
    print(f"\n  Max F1 bod: Návratnosť={best_f1_recall:.3f},␣
 ↪Presnosť={best_f1_precision:.3f}")
    print(f"  Max Presnosť bod: Návratnosť={best_prec_recall:.3f},␣
 ↪Presnosť={best_prec_precision:.3f}")

# General eval function
def evaluate_model(model_type, model_path, image_dir, label_dir, input_size,␣
 ↪threshold, model_name):
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Používam zariadenie: {device}")

    # Load Model
    if model_type == "cnn":
        model = CNNModel().to(device)
    else:
        model = timm.create_model(model_type, pretrained=False, num_classes=1).
 ↪to(device)

        # ConvNeXt-Small
        if model_type == "convnext_small":
            model.classifier = nn.Sequential(
                nn.Dropout(0.5),
                nn.Linear(model.num_features, 1)
            )
```

```
    # Load Mkdel Weights
    model.load_state_dict(torch.load(model_path, map_location=device))
    model.eval()

    # Transformations
    transform = transforms.Compose([
        transforms.Resize(input_size),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.
↪225]),
    ])

    # List of Images and Labels
    image_files = sorted([
        f for f in os.listdir(image_dir)
        if os.path.isfile(os.path.join(image_dir, f)) and f.lower().endswith(('.
↪jpg', '.jpeg', '.png'))
    ])
    label_files = sorted([
        f for f in os.listdir(label_dir)
        if os.path.isfile(os.path.join(label_dir, f)) and f.lower().endswith('.
↪txt')
    ])

    # Output lists
    true_labels = []
    predicted_labels = []
    confidences = []

    for img_name, label_name in zip(image_files, label_files):
        img_path = os.path.join(image_dir, img_name)
        image = Image.open(img_path).convert('RGB')
        image = transform(image).unsqueeze(0).to(device)

        label_path = os.path.join(label_dir, label_name)
        with open(label_path, 'r') as f:
            content = f.read().strip()
            label = 0 if content == '' else 1

        with torch.no_grad():
            output = model(image)
            confidence = torch.sigmoid(output).item()
            predicted_label = 1 if confidence >= threshold else 0

        true_labels.append(label)
        predicted_labels.append(predicted_label)
```

```
        confidences.append(confidence)

    df_preds = pd.DataFrame({
        'true_label': true_labels,
        'predicted_label': predicted_labels,
        'confidence': confidences
    })

    print(df_preds.head())
    evaluate_and_plot_plotly(df_preds, model_name, threshold)
```

Vlastná CNN

[4]:
```
# SE Block
class SEBlock(nn.Module):
    def __init__(self, in_channels, reduction=16):
        super(SEBlock, self).__init__()
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(in_channels, in_channels // reduction),
            nn.ReLU(),
            nn.Linear(in_channels // reduction, in_channels),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        y = self.pool(x).view(b, c)
        y = self.fc(y).view(b, c, 1, 1)
        return x * y

# Vlastná CNN s SE blokmi
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.se1 = SEBlock(32)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.se2 = SEBlock(64)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
        self.se3 = SEBlock(128)
        self.dropout = nn.Dropout(0.3)

        sample_input = torch.randn(1, 3, 320, 480)
        with torch.no_grad():
            feature_size = self._get_feature_map_size(sample_input)
```

```
        self.fc1 = nn.Linear(feature_size, 128)
        self.fc2 = nn.Linear(128, 1)

    def _get_feature_map_size(self, x):
        x = torch.relu(self.conv1(x))
        x = self.se1(x)
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv2(x))
        x = self.se2(x)
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv3(x))
        x = self.se3(x)
        x = torch.max_pool2d(x, 2)
        return x.view(1, -1).size(1)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.se1(x)
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv2(x))
        x = self.se2(x)
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv3(x))
        x = self.se3(x)
        x = torch.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = self.dropout(x)
        x = torch.relu(self.fc1(x))
        return torch.sigmoid(self.fc2(x))

evaluate_model(
    model_type="cnn",
    model_path="/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/
 ↪meteor_detection_cnn_model_v4.pth",
    image_dir="/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test/images",
    label_dir="/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test/labels",
    input_size=(480, 480),
    threshold=0.54,
    model_name="Vlastná CNN"
)
```

```
Používam zariadenie: cuda
   true_label  predicted_label  confidence
0           0                1    0.676046
```

```
1           0               1    0.562982
2           0               1    0.667421
3           0               1    0.676724
4           0               1    0.681732

 Výsledky pre prah 0.54:
 TP (Správne meteory): 552
 TN (Správne nemeteory): 17
 FP (Falošné meteory): 307
 FN (Nezachytené meteory): 48
 Presnosť (Precision): 0.6426
 Návratnosť (Recall): 0.9200
 F1-Skóre: 0.7567
 Presnosť klasifikácie (Accuracy): 0.6158
 Prekrytie distribúcie (Overlap coefficient): 68.64%


 Max F1 bod: Návratnosť=1.000, Presnosť=0.649
 Max Presnosť bod: Návratnosť=0.000, Presnosť=1.000
```

EfficientNet-B4

[5]:
```
evaluate_model(
    model_type="efficientnet_b4",
    model_path="/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/
 ↪best_efficientNetB4_model_v6.pth",
    image_dir="/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test/images",
    label_dir="/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test/labels",
    input_size=(720, 720),
    threshold=0.25,
    model_name="EfficientNet-B4"
)
```

```
Používam zariadenie: cuda
   true_label  predicted_label  confidence
0           0               0    0.075326
1           0               0    0.110707
2           0               0    0.022697
3           0               0    0.058285
4           0               0    0.025138

 Výsledky pre prah 0.25:
 TP (Správne meteory): 575
 TN (Správne nemeteory): 278
 FP (Falošné meteory): 46
 FN (Nezachytené meteory): 25
```

```
Presnosť (Precision): 0.9259
Návratnosť (Recall): 0.9583
F1-Skóre: 0.9419
Presnosť klasifikácie (Accuracy): 0.9232
Prekrytie distribúcie (Overlap coefficient): 18.36%


Max F1 bod: Návratnosť=0.970, Presnosť=0.921
Max Presnosť bod: Návratnosť=0.310, Presnosť=1.000
```

ConvNext-Small

[6]:
```
evaluate_model(
    model_type="convnext_small",
    model_path="/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ConvNext-Small/datasetv2_best_convnext_model_v65_epocha2.
 ↪pth",
    image_dir="/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test/images",
    label_dir="/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test/labels",
    input_size=(720, 720),
    threshold=0.5453,
    model_name="ConvNeXt-Small"
)
```

```
Používam zariadenie: cuda
   true_label  predicted_label  confidence
0           0                0    0.128172
1           0                0    0.212369
2           0                0    0.276863
3           0                0    0.155673
4           0                0    0.211172

Výsledky pre prah 0.55:
TP (Správne meteory): 596
TN (Správne nemeteory): 284
FP (Falošné meteory): 40
FN (Nezachytené meteory): 4
Presnosť (Precision): 0.9371
Návratnosť (Recall): 0.9933
F1-Skóre: 0.9644
Presnosť klasifikácie (Accuracy): 0.9524
Prekrytie distribúcie (Overlap coefficient): 12.94%


Max F1 bod: Návratnosť=0.995, Presnosť=0.936
Max Presnosť bod: Návratnosť=0.413, Presnosť=1.000
```

Yolov11

[7]:
```
model_path = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/najlepsie_modely/yolov11_vacsie_obrazky.pt"
image_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test_yolo/images"
label_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test_yolo/labels"

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Používam zariadenie: {device}")

model = YOLO(model_path).to(device)
model.eval()

image_files = sorted([
    f for f in os.listdir(image_dir)
    if os.path.isfile(os.path.join(image_dir, f)) and f.lower().endswith(('.
 ↪jpg', '.jpeg', '.png'))
])

label_files = sorted([
    f for f in os.listdir(label_dir)
    if os.path.isfile(os.path.join(label_dir, f)) and f.lower().endswith('.txt')
])

true_labels = []
confidences = []
predicted_labels = []

# Threshold
threshold = 0.275

# Evaluation on the test set
for img_name, label_name in tqdm(zip(image_files, label_files),␣
 ↪total=len(image_files), desc="Spracovanie obrázkov"):
    # if label file is not empty -> 1 (meteor), otherwise 0 (non-meteor)
    label_path = os.path.join(label_dir, label_name)
    with open(label_path, 'r') as f:
        content = f.read().strip()
        label = 1 if content else 0

    img_path = os.path.join(image_dir, img_name)

    # Prediction
    with torch.no_grad():
        results = model(img_path, verbose=False)  # prediction
        detections = results[0].boxes  # detection
```

```
    # Confidence logic: if at least 1 detection, take max confidence, otherwise␣
 ↪0
    if len(detections) > 0:
        max_confidence = detections.conf.max().item()
    else:
        max_confidence = 0.0

    # Classification by threshold
    predicted_label = 1 if max_confidence >= threshold else 0

    true_labels.append(label)
    confidences.append(max_confidence)
    predicted_labels.append(predicted_label)

df_preds = pd.DataFrame({
    'true_label': true_labels,
    'confidence': confidences,
    'predicted_label': predicted_labels
})

print(df_preds.head())

evaluate_and_plot_plotly(df_preds, "Yolov11", threshold=threshold)
```

```
Používam zariadenie: cuda

Spracovanie obrázkov: 100%|      | 924/924 [00:53<00:00, 17.14it/s]

   true_label  confidence  predicted_label
0           0         0.0                0
1           0         0.0                0
2           0         0.0                0
3           0         0.0                0
4           0         0.0                0

 Výsledky pre prah 0.28:
 TP (Správne meteory): 580
 TN (Správne nemeteory): 300
 FP (Falošné meteory): 24
 FN (Nezachytené meteory): 20
 Presnosť (Precision): 0.9603
 Návratnosť (Recall): 0.9667
 F1-Skóre: 0.9635
 Presnosť klasifikácie (Accuracy): 0.9524
 Prekrytie distribúcie (Overlap coefficient): 10.79%


 Max F1 bod: Návratnosť=0.972, Presnosť=0.957
 Max Presnosť bod: Návratnosť=0.128, Presnosť=1.000
```

Yolov12

[8]:
```
model_path = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/najlepsie_modely/yolov12_vacsie_obrazky.pt"
image_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test_yolo/images"
label_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪najlepsie_modely/vyhodnotenie/test_yolo/labels"

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Používam zariadenie: {device}")

model = YOLO(model_path).to(device)
model.eval()

image_files = sorted([
    f for f in os.listdir(image_dir)
    if os.path.isfile(os.path.join(image_dir, f)) and f.lower().endswith(('.
 ↪jpg', '.jpeg', '.png'))
])

label_files = sorted([
    f for f in os.listdir(label_dir)
    if os.path.isfile(os.path.join(label_dir, f)) and f.lower().endswith('.txt')
])

true_labels = []
confidences = []
predicted_labels = []

# Threshold
threshold = 0.3201

for img_name, label_name in tqdm(zip(image_files, label_files),␣
 ↪total=len(image_files), desc="Spracovanie obrázkov"):

    label_path = os.path.join(label_dir, label_name)
    with open(label_path, 'r') as f:
        content = f.read().strip()
        label = 1 if content else 0  # empty = non-meteor

    img_path = os.path.join(image_dir, img_name)

    with torch.no_grad():
        results = model(img_path, verbose=False)
        detections = results[0].boxes
```

```
    if len(detections) > 0:
        max_confidence = detections.conf.max().item()
    else:
        max_confidence = 0.0

    predicted_label = 1 if max_confidence >= threshold else 0

    true_labels.append(label)
    confidences.append(max_confidence)
    predicted_labels.append(predicted_label)

df_preds = pd.DataFrame({
    'true_label': true_labels,
    'confidence': confidences,
    'predicted_label': predicted_labels
})

print(df_preds.head())

evaluate_and_plot_plotly(df_preds, "Yolov12", threshold=threshold)
```

```
Používam zariadenie: cuda
Spracovanie obrázkov: 100%|      | 924/924 [01:37<00:00,  9.49it/s]
   true_label  confidence  predicted_label
0           0         0.0                0
1           0         0.0                0
2           0         0.0                0
3           0         0.0                0
4           0         0.0                0

 Výsledky pre prah 0.32:
 TP (Správne meteory): 571
 TN (Správne nemeteory): 302
 FP (Falošné meteory): 22
 FN (Nezachytené meteory): 29
 Presnosť (Precision): 0.9629
 Návratnosť (Recall): 0.9517
 F1-Skóre: 0.9573
 Presnosť klasifikácie (Accuracy): 0.9448
 Prekrytie distribúcie (Overlap coefficient): 11.74%



 Max F1 bod: Návratnosť=0.970, Presnosť=0.949
 Max Presnosť bod: Návratnosť=0.288, Presnosť=1.000
```

1.1.5 Učenie súborom metód

Optimalizácia výkonu modelu vzhľadom na minimalizáciu falošných klasifikácií a času predikovania

[10]:
```
import os
import torch
import pandas as pd
import time
from tqdm import tqdm
from PIL import Image
from torchvision import transforms
from ultralytics import YOLO
import timm
import torch.nn as nn
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.subplots as sp
from sklearn.metrics import confusion_matrix, precision_score, recall_score,␣
 ↪f1_score, accuracy_score, precision_recall_curve
```

[ ]:
```
# === PATHS ===
test_images_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/vyhodnotenie/test/images"
model_path_convnext = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ConvNext-Small/datasetv2_best_convnext_model_v65_epocha2.
 ↪pth"
model_path_efficientnetb4 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/
 ↪best_efficientNetB4_model_v6.pth"
model_path_yolov12 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov12_vacsie_obrazky.
 ↪pt"
model_path_yolov11 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov11_vacsie_obrazky.
 ↪pt"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# === IMAGES ===
image_files = sorted([
    f for f in os.listdir(test_images_dir)
    if os.path.isfile(os.path.join(test_images_dir, f)) and f.lower().
 ↪endswith(('.jpg', '.jpeg', '.png'))
])
image_paths = [os.path.join(test_images_dir, f) for f in image_files]

# === TRANSFORMATIONS ===
```

```
transform = transforms.Compose([
    transforms.Resize((720, 720)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# === MODELS ===
convnext_model = timm.create_model("convnext_small", pretrained=False,␣
 ↪num_classes=1).to(device)
convnext_model.classifier = nn.Sequential(nn.Dropout(0.5), nn.
 ↪Linear(convnext_model.num_features, 1))
convnext_model.load_state_dict(torch.load(model_path_convnext,␣
 ↪map_location=device))
convnext_model.eval()

efficientnet_model = timm.create_model("efficientnet_b4", pretrained=False,␣
 ↪num_classes=1).to(device)
efficientnet_model.load_state_dict(torch.load(model_path_efficientnetb4,␣
 ↪map_location=device))
efficientnet_model.eval()

yolo_model_11 = YOLO(model_path_yolov11).to(device)
yolo_model_12 = YOLO(model_path_yolov12).to(device)
yolo_model_11.eval()
yolo_model_12.eval()

# === MEASURE PREDICTION TIME ===
def measure_time(predict_fn, model_name):
    times = []
    for img_path in tqdm(image_paths, desc=f"Predikcia {model_name}"):
        start = time.time()
        _ = predict_fn(img_path)
        end = time.time()
        times.append(end - start)
    return sum(times)

def predict_convnext(image_path):
    image = Image.open(image_path).convert('RGB')
    image = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        return torch.sigmoid(convnext_model(image)).item()

def predict_efficientnet(image_path):
    image = Image.open(image_path).convert('RGB')
    image = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        return torch.sigmoid(efficientnet_model(image)).item()
```

```
def predict_yolo(image_path, model):
    with torch.no_grad():
        results = model(image_path, verbose=False)
        detections = results[0].boxes
    confidences = [det[4] for det in detections.data.cpu().numpy()]
    return max(confidences) if confidences else 0.0

# === TOTAL TIME OF PREDICTION BY MODEL ===
time_yolov11 = measure_time(lambda x: predict_yolo(x, yolo_model_11), "YOLOv11")
time_yolov12 = measure_time(lambda x: predict_yolo(x, yolo_model_12), "YOLOv12")
time_convnext = measure_time(predict_convnext, "ConvNeXt-Small")
time_effnet = measure_time(predict_efficientnet, "EfficientNet-B4")

# === CALCULATE COMBINATIONS ===
total_images = len(image_paths)
avg_times = {
    "YOLOv11": time_yolov11 / total_images,
    "YOLOv12": time_yolov12 / total_images,
    "ConvNext-Small": time_convnext / total_images,
    "EfficientNet-B4": time_effnet / total_images
}

# === ENSEMBLE CONFIGS ===
combinations = {
    "YOLOv11": ["YOLOv11"],
    "Yv11 + Yv12 + CnvX + EfN": ["YOLOv11", "YOLOv12", "ConvNext-Small",␣
 ↪"EfficientNet-B4"],
    "Yv11 + CnvX + EfN": ["YOLOv11", "ConvNext-Small", "EfficientNet-B4"],
    "Yv11 + Yv12 + CnvX": ["YOLOv11", "YOLOv12", "ConvNext-Small"],
    "Yv12 + CnvX": ["YOLOv12", "ConvNext-Small"]
}

# === OUTPUT TAB ===
rows = []
for name, models in combinations.items():
    total_time = sum([avg_times[m] for m in models]) * total_images
    rows.append({"Kombinácia": name, "Modely": models, "Čas predikcie (s)":␣
 ↪round(total_time, 2)})

df_result = pd.DataFrame(rows)
df_result.to_csv("ensemble_prediction_times.csv", index=False)
print("  Výsledná tabuľka uložená do ensemble_prediction_times.csv")
print(df_result)
```

[11]:

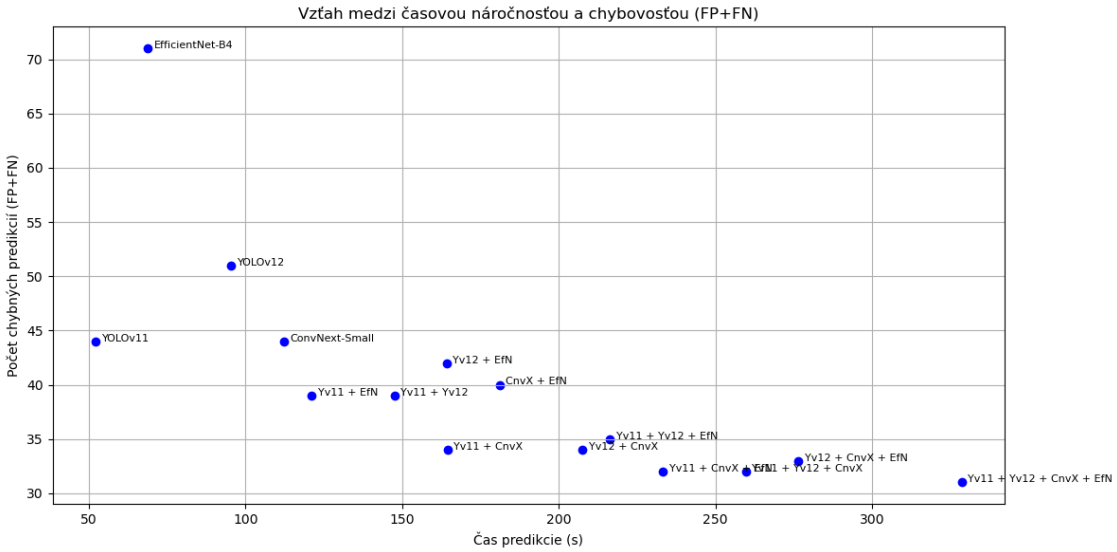
```
# === DIRS ===
test_images_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/vyhodnotenie/test/images"
model_path_convnext = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ConvNext-Small/datasetv2_best_convnext_model_v65_epocha2.
 ↪pth"
model_path_efficientnetb4 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/
 ↪best_efficientNetB4_model_v6.pth"
model_path_yolov12 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov12_vacsie_obrazky.
 ↪pt"
model_path_yolov11 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov11_vacsie_obrazky.
 ↪pt"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# === IMAGES ===
image_files = sorted([
    f for f in os.listdir(test_images_dir)
    if os.path.isfile(os.path.join(test_images_dir, f)) and f.lower().
 ↪endswith(('.jpg', '.jpeg', '.png'))
])
image_paths = [os.path.join(test_images_dir, f) for f in image_files]

# === TRANSFORMATIONS ===
transform = transforms.Compose([
    transforms.Resize((720, 720)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# === MODELS ===
convnext_model = timm.create_model("convnext_small", pretrained=False,␣
 ↪num_classes=1).to(device)
convnext_model.classifier = nn.Sequential(nn.Dropout(0.5), nn.
 ↪Linear(convnext_model.num_features, 1))
convnext_model.load_state_dict(torch.load(model_path_convnext,␣
 ↪map_location=device))
convnext_model.eval()

efficientnet_model = timm.create_model("efficientnet_b4", pretrained=False,␣
 ↪num_classes=1).to(device)
efficientnet_model.load_state_dict(torch.load(model_path_efficientnetb4,␣
 ↪map_location=device))
```

```
efficientnet_model.eval()

yolo_model_11 = YOLO(model_path_yolov11).to(device)
yolo_model_12 = YOLO(model_path_yolov12).to(device)
yolo_model_11.eval()
yolo_model_12.eval()

# === MEASURE TIME ===
def measure_time(predict_fn, model_name):
    times = []
    for img_path in tqdm(image_paths, desc=f"Predikcia {model_name}"):
        start = time.time()
        _ = predict_fn(img_path)
        end = time.time()
        times.append(end - start)
    return sum(times)

def predict_convnext(image_path):
    image = Image.open(image_path).convert('RGB')
    image = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        return torch.sigmoid(convnext_model(image)).item()

def predict_efficientnet(image_path):
    image = Image.open(image_path).convert('RGB')
    image = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        return torch.sigmoid(efficientnet_model(image)).item()

def predict_yolo(image_path, model):
    with torch.no_grad():
        results = model(image_path, verbose=False)
        detections = results[0].boxes
    confidences = [det[4] for det in detections.data.cpu().numpy()]
    return max(confidences) if confidences else 0.0

total_images = len(image_paths)
time_yolov11 = measure_time(lambda x: predict_yolo(x, yolo_model_11), "YOLOv11")
time_yolov12 = measure_time(lambda x: predict_yolo(x, yolo_model_12), "YOLOv12")
time_convnext = measure_time(predict_convnext, "ConvNeXt-Small")
time_effnet = measure_time(predict_efficientnet, "EfficientNet-B4")

avg_times = {
    "YOLOv11": time_yolov11 / total_images,
    "YOLOv12": time_yolov12 / total_images,
    "ConvNext-Small": time_convnext / total_images,
    "EfficientNet-B4": time_effnet / total_images
```

```
}

# === COMBINATIONS ===
combinations_fpfn = {
    "Yv11 + Yv12 + CnvX + EfN": (["YOLOv11", "YOLOv12", "ConvNext-Small",␣
 ↪"EfficientNet-B4"], 31),
    "Yv11 + CnvX + EfN": (["YOLOv11", "ConvNext-Small", "EfficientNet-B4"], 32),
    "Yv11 + Yv12 + CnvX": (["YOLOv11", "YOLOv12", "ConvNext-Small"], 32),
    "Yv12 + CnvX + EfN": (["YOLOv12", "ConvNext-Small", "EfficientNet-B4"], 33),
    "Yv12 + CnvX": (["YOLOv12", "ConvNext-Small"], 34),
    "Yv11 + CnvX": (["YOLOv11", "ConvNext-Small"], 34),
    "Yv11 + Yv12 + EfN": (["YOLOv11", "YOLOv12", "EfficientNet-B4"], 35),
    "Yv11 + EfN": (["YOLOv11", "EfficientNet-B4"], 39),
    "Yv11 + Yv12": (["YOLOv11", "YOLOv12"], 39),
    "CnvX + EfN": (["ConvNext-Small", "EfficientNet-B4"], 40),
    "Yv12 + EfN": (["YOLOv12", "EfficientNet-B4"], 42),
    "YOLOv11": (["YOLOv11"], 44),
    "YOLOv12": (["YOLOv12"], 51),
    "ConvNext-Small": (["ConvNext-Small"], 44),
    "EfficientNet-B4": (["EfficientNet-B4"], 71),
}

# === OUTPUT TABLE ===
rows = []
for name, (models, fpfn) in combinations_fpfn.items():
    total_time = sum([avg_times[m] for m in models]) * total_images
    rows.append({
        "Kombinácia": name,
        "Modely": models,
        "Čas predikcie (s)": round(total_time, 2),
        "FP+FN": fpfn
    })

df_result = pd.DataFrame(rows)
df_result.to_csv("ensemble_all_combinations_times_and_errors.csv", index=False)

# === PLOT ===
df_valid = df_result.dropna(subset=["Čas predikcie (s)", "FP+FN"])
plt.figure(figsize=(12, 6))
plt.scatter(df_valid["Čas predikcie (s)"], df_valid["FP+FN"], c="blue")
for _, row in df_valid.iterrows():
    plt.text(row["Čas predikcie (s)"] + 2, row["FP+FN"], row["Kombinácia"],␣
 ↪fontsize=8)
plt.title("Vzťah medzi časovou náročnosťou a chybovosťou (FP+FN)")
plt.xlabel("Čas predikcie (s)")
plt.ylabel("Počet chybných predikcií (FP+FN)")
plt.grid(True)
```

```
plt.tight_layout()
plt.savefig("cas_vs_fpfn_all_combinations.png")
plt.show()
```

```
Predikcia YOLOv11: 100%|      | 924/924 [00:52<00:00, 17.58it/s]
Predikcia YOLOv12: 100%|      | 924/924 [01:35<00:00,  9.63it/s]
Predikcia ConvNeXt-Small: 100%|      | 924/924 [01:53<00:00,  8.17it/s]
Predikcia EfficientNet-B4: 100%|      | 924/924 [01:09<00:00, 13.33it/s]
```



[12]:
```
df = pd.read_csv("ensemble_all_combinations_times_and_errors.csv")

model_map = {
    "YOLOv11": "Yv11",
    "YOLOv12": "Yv12",
    "ConvNext-Small": "CnvX",
    "EfficientNet-B4": "EfN",
}

custom_blues_r = [
    [0.0, 'rgb(198,219,239)'],
    [0.25, 'rgb(158,202,225)'],
    [0.5, 'rgb(107,174,214)'],
    [0.75, 'rgb(49,130,189)'],
    [1.0, 'rgb(8,81,156)']
]

def shorten_name(name):
    for long, short in model_map.items():
        name = name.replace(long, short)
```

```
    return name

df["Skratka"] = df["Kombinácia"].apply(shorten_name)
df["Skóre výkonu"] = df["FP+FN"] + df["Čas predikcie (s)"] * 0.075

# Best point = the lowest combined score
top_idx = df["Skóre výkonu"].idxmin()

text_positions = []
for i, row in df.iterrows():
    if row["Skratka"] == "Yv11 + Yv12":
        text_positions.append("bottom left")
    elif row["Skratka"] == "CnvX + EfN":
        text_positions.append("top right")
    elif row["Skratka"] == "Yv11 + CnvX + EfN":
        text_positions.append("bottom left")
    elif row["Skratka"] == "Yv12 + CnvX + EfN":
        text_positions.append("top right")
    elif row["Skratka"] == "Yv11 + Yv12 + EfN":
        text_positions.append("top right")
    elif row["Skratka"] == "Yv11 + Yv12 + CnvX + EfN":
        text_positions.append("bottom left")
    else:
        text_positions.append("top left")

# All points (without the best)
scatter_data = []

scatter_data.append(go.Scatter(
    x=df.loc[df.index != top_idx, "Čas predikcie (s)"],
    y=df.loc[df.index != top_idx, "FP+FN"],
    mode='markers+text',
    marker=dict(
        color=df.loc[df.index != top_idx, "Skóre výkonu"],
        colorscale= "Blues_r",
        size=15,
        line=dict(color='black', width=1),
        showscale=False
    ),
    text=df.loc[df.index != top_idx, "Skratka"],
    textposition=[pos for i, pos in enumerate(text_positions) if i != top_idx],
    textfont=dict(size=15),
    name="Modely"
))

# Best point - yellow
scatter_data.append(go.Scatter(
```

```
    x=[df.loc[top_idx, "Čas predikcie (s)"]],
    y=[df.loc[top_idx, "FP+FN"]],
    mode='markers+text',
    marker=dict(
        color='#FFD700',
        size=15,
        line=dict(color='black', width=2)
    ),
    text=[df.loc[top_idx, "Skratka"]],
    textposition=[text_positions[top_idx]],
    textfont=dict(size=15),
    name="Najlepší kompromis"
))

# Layout
layout = go.Layout(
    title=dict(
        text="Vzťah medzi časovou náročnosťou a chybovosťou modelov",
        x=0.5, xanchor='center',
        font=dict(size=20)
    ),
    plot_bgcolor='#F5F5F5',
    paper_bgcolor='white',
    font=dict(size=13, color='black'),
    margin=dict(l=60, r=60, t=80, b=60),
    xaxis=dict(
        title="Čas predikcie (s)",
        range=[0, 340],
        dtick=20,
        showgrid=True,
        gridcolor='white'
    ),
    yaxis=dict(
        title="Počet chybných predikcií (FP+FN)",
        range=[0, 75],
        dtick=5,
        showgrid=True,
        gridcolor='white'
    ),
    autosize=False,
    width=1200,
    height=850,
)

fig = go.Figure(data=scatter_data, layout=layout)
fig.show()
```

[ ]:
```
# Zoradené skóre podľa výkonnosti kombinovaných modelov
df_sorted = df.sort_values(by="Skóre výkonu", ascending=True)

top_8_compromises = df_sorted.head(8)

print("Najlepšie 8 kompromisy:")
print(top_8_compromises[["Skratka", "Skóre výkonu", "Čas predikcie (s)",␣
 ↪"FP+FN"]])
```

Optimalizácia váh a rozhodovacieho prahu kombinovaného modelu

[17]:
```
import os
import random
import numpy as np
import pandas as pd
import timm
import torch
import torch.nn as nn

from torchvision import transforms
from tqdm import tqdm
from PIL import Image
from ultralytics import YOLO

import plotly.graph_objects as go
import plotly.subplots as sp

from sklearn.metrics import classification_report, confusion_matrix,␣
 ↪precision_score, recall_score, f1_score, accuracy_score,␣
 ↪precision_recall_curve

from skopt import gp_minimize
from skopt.space import Real
from skopt.utils import use_named_args

from deap import base, creator, tools, algorithms
```

[ ]:
```
test_images_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/vyhodnotenie/test/images"
test_labels_dir_classification = "/home/jovyan/data/lightning/LiviaMurankova/
 ↪new/namnozene_meteory/najlepsie_modely/vyhodnotenie/test/labels"
test_labels_dir_detection = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/vyhodnotenie/test_yolo/labels"

model_path_convnext = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ConvNext-Small/datasetv2_best_convnext_model_v65_epocha2.
 ↪pth"
```

```
model_path_efficientnetb4 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/
 ↪best_efficientNetB4_model_v6.pth"
model_path_yolov12 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov12_vacsie_obrazky.
 ↪pt"
model_path_yolov11 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov11_vacsie_obrazky.
 ↪pt"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Používam zariadenie: {device}")

# ======= 1. CONVNEXT-SMALL =======
# Transformations according to training ConvNext-Small
transform = transforms.Compose([
    transforms.Resize((720, 720)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

convnext_model = timm.create_model("convnext_small", pretrained=False,␣
 ↪num_classes=1).to(device)
convnext_model.classifier = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(convnext_model.num_features, 1)
)
convnext_model.load_state_dict(torch.load(model_path_convnext,␣
 ↪map_location=device))
convnext_model.eval()

def predict_convnext(image_path, threshold=0.65):
    image = Image.open(image_path).convert('RGB')
    image = transform(image).unsqueeze(0).to(device)  # Batch dim

    with torch.no_grad():
        output = torch.sigmoid(convnext_model(image))  # prob. of meteor
        probability = output.item()

    #predicted_label = 1 if probability >= threshold else 0  # Binárna␣
 ↪klasifikácia podľa prahu

    return probability

# ======= 2. EFFICIENTNET-B4 =======
```

```
efficientnet_model = timm.create_model("efficientnet_b4", pretrained=False,␣
 ↪num_classes=1).to(device)
efficientnet_model.load_state_dict(torch.load(model_path_efficientnetb4,␣
 ↪map_location=device))
efficientnet_model.eval()

def predict_efficientnet(image_path, threshold=0.5):
    image = Image.open(image_path).convert('RGB')
    image = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        output = torch.sigmoid(efficientnet_model(image))
        return output.item()

# ======= 3. YOLOV11 a YOLOV12 =======
yolo_model_11 = YOLO(model_path_yolov11).to(device)
yolo_model_12 = YOLO(model_path_yolov12).to(device)
yolo_model_11.eval()
yolo_model_12.eval()

def predict_yolo(image_path, model, threshold=0.3):
    with torch.no_grad():
        results = model(image_path, verbose=False)
        detections = results[0].boxes  # Bounding boxes and confidence

    confidences = [det[4] for det in detections.data.cpu().numpy() if det[4] >␣
 ↪threshold]

    return max(confidences) if confidences else 0.0

# ======= 4. PROCESS LABELS =======
def get_true_label(label_path):
    if os.path.exists(label_path):
        with open(label_path, "r") as f:
            content = f.read().strip()
        return 1 if content.startswith("0") else 0
    return 0

# ======= 5. CREATE DATAFRAME =======
results = []

image_files = sorted([
    f for f in os.listdir(test_images_dir)
    if os.path.isfile(os.path.join(test_images_dir, f)) and f.lower().
 ↪endswith(('.jpg', '.jpeg', '.png'))
])

label_files_classification = sorted([
```

```
    f for f in os.listdir(test_labels_dir_classification)
    if os.path.isfile(os.path.join(test_labels_dir_classification, f)) and f.
 ↪lower().endswith('.txt')
])

label_files_detection = sorted([
    f for f in os.listdir(test_labels_dir_detection)
    if os.path.isfile(os.path.join(test_labels_dir_detection, f)) and f.lower().
 ↪endswith('.txt')
])

# Process test images
for img_name in tqdm(image_files, desc="Spracovanie obrázkov"):
    img_path = os.path.join(test_images_dir, img_name)
    label_path_classification = os.path.join(test_labels_dir_classification,␣
 ↪img_name.replace(".jpg", ".txt"))
    label_path_detection = os.path.join(test_labels_dir_detection, img_name.
 ↪replace(".jpg", ".txt"))

    true_label = get_true_label(label_path_classification)

    pred_convnext = predict_convnext(img_path, threshold=0)              #␣
 ↪threshold=0.5
    pred_eff = predict_efficientnet(img_path, threshold=0)
    pred_yolo11 = predict_yolo(img_path, yolo_model_11, threshold=0)    #␣
 ↪threshold=0.275
    pred_yolo12 = predict_yolo(img_path, yolo_model_12, threshold=0)   #␣
 ↪threshold=0.3201

    results.append([img_name, true_label, pred_convnext, pred_yolo11,␣
 ↪pred_yolo12, pred_eff])

# ======= 5. SAVE IN CSV =======
df = pd.DataFrame(results, columns=["image_name", "true_label",␣
 ↪"prediction_ConvNexts", "prediction_yolov11", "prediction_yolov12",␣
 ↪"prediction_EfficientNetB4"])
df.to_csv("all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv",␣
 ↪index=False)

print(" CSV súbor `all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.
 ↪csv` bol úspešne vytvorený.")
```

Genetický algoritmus použitý na heuristickú optimalizáciu váh modelov a rozhodovacieho prahu kombinovaného modelu

[3]:
```
# 4 modely
# not dependant on the order of models in weighing

df = pd.read_csv("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv")

def compute_fp_fn(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return fp + fn

def evaluate(individual):
    weights = np.array(individual[:4])
    weights /= np.sum(weights)
    w1, w2, w3, w4 = weights
    threshold = individual[4]

    df["ensemble_probability"] = (
        df["prediction_yolov11"] * w1 +
        df["prediction_yolov12"] * w2 +
        df["prediction_ConvNexts"] * w3 +
        df["prediction_EfficientNetB4"] * w4
    )

    df["ensemble_prediction"] = (df["ensemble_probability"] >= threshold).
 ↪astype(int)
    score = compute_fp_fn(df["true_label"], df["ensemble_prediction"])
    return score,

creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

toolbox = base.Toolbox()
toolbox.register("individual", lambda: creator.Individual([
    random.random(), random.random(), random.random(), random.random(),    #␣
 ↪weights
    random.uniform(0.1, 0.9)                                               #␣
 ↪threshold
]))
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.15, indpb=0.5)
toolbox.register("select", tools.selTournament, tournsize=3)

population = toolbox.population(n=400)
```

```
NGEN = 150
CXPB = 0.8
MUTPB = 0.3

for gen in range(NGEN):
    offspring = algorithms.varAnd(population, toolbox, cxpb=CXPB, mutpb=MUTPB)
    fits = toolbox.map(toolbox.evaluate, offspring)
    for fit, ind in zip(fits, offspring):
        ind.fitness.values = fit
    elite = tools.selBest(population, k=5)
    population = toolbox.select(offspring, k=len(population) - 5) + elite

top_ind = tools.selBest(population, k=1)[0]
top_weights = np.array(top_ind[:4]) / np.sum(top_ind[:4])
w1, w2, w3, w4 = top_weights
threshold = top_ind[4]
fp_fn = evaluate([w1, w2, w3, w4, threshold])[0]

print("\n Najlepšie nájdené váhy a prah (bez podmienky poradia):")
print(f"yolov11          = {w1:.4f}")
print(f"yolov12          = {w2:.4f}")
print(f"convnexts        = {w3:.4f}")
print(f"efficientnetb4   = {w4:.4f}")
print(f"threshold        = {threshold:.3f}")
print(f"fp + fn          = {fp_fn}")
```

[ ]:
```
# 3 modely
df = pd.read_csv("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv")

def compute_fp_fn(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return fp + fn

def evaluate(individual):
    weights = np.array(individual[:3])
    weights /= np.sum(weights)
    w1, w2, w3 = weights
    threshold = individual[3]

    df["ensemble_probability"] = (
        df["prediction_yolov11"] * w1 +
        df["prediction_yolov12"] * w2 +
        df["prediction_ConvNexts"] * w3
    )
```

```
    df["ensemble_prediction"] = (df["ensemble_probability"] >= threshold).
 ↪astype(int)
    score = compute_fp_fn(df["true_label"], df["ensemble_prediction"])
    return score,

creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

toolbox = base.Toolbox()
toolbox.register("individual", lambda: creator.Individual([
    random.random(), random.random(), random.random(), random.random(),
    random.uniform(0.1, 0.9)
]))
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.15, indpb=0.5)
toolbox.register("select", tools.selTournament, tournsize=3)

population = toolbox.population(n=400)
NGEN = 150
CXPB = 0.8
MUTPB = 0.3

for gen in range(NGEN):
    offspring = algorithms.varAnd(population, toolbox, cxpb=CXPB, mutpb=MUTPB)
    fits = toolbox.map(toolbox.evaluate, offspring)
    for fit, ind in zip(fits, offspring):
        ind.fitness.values = fit
    elite = tools.selBest(population, k=5)
    population = toolbox.select(offspring, k=len(population) - 5) + elite

top_ind = tools.selBest(population, k=1)[0]
top_weights = np.array(top_ind[:3]) / np.sum(top_ind[:3])
w1, w2, w3 = top_weights
threshold = top_ind[3]
fp_fn = evaluate([w1, w2, w3, threshold])[0]

print("\n  Najlepšie nájdené váhy a prah (bez podmienky poradia):")
print(f"yolov11         = {w1:.4f}")
print(f"yolov12         = {w2:.4f}")
print(f"convnexts       = {w3:.4f}")
print(f"threshold       = {threshold:.3f}")
print(f"fp + fn         = {fp_fn}")
```

[ ]:
```
# 2 modely

df = pd.read_csv("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv")

def compute_fp_fn(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return fp + fn

def evaluate(individual):
    weights = np.array(individual[:2])
    weights /= np.sum(weights)

    w1, w2 = weights
    threshold = individual[2]

    df["ensemble_probability"] = (
        df["prediction_yolov11"] * w1 +
        df["prediction_ConvNexts"] * w2
    )

    df["ensemble_prediction"] = (df["ensemble_probability"] >= threshold).
 ↪astype(int)
    score = compute_fp_fn(df["true_label"], df["ensemble_prediction"])
    return score,

creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

toolbox = base.Toolbox()
toolbox.register("individual", lambda: creator.Individual([
    random.random(),
    random.random(),
    random.uniform(0.1, 0.9)        # threshold
]))
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.15, indpb=0.5)
toolbox.register("select", tools.selTournament, tournsize=3)

population = toolbox.population(n=300)
NGEN = 100
CXPB = 0.8
MUTPB = 0.3
```

```
for gen in range(NGEN):
    offspring = algorithms.varAnd(population, toolbox, cxpb=CXPB, mutpb=MUTPB)
    fits = toolbox.map(toolbox.evaluate, offspring)
    for fit, ind in zip(fits, offspring):
        ind.fitness.values = fit
    elite = tools.selBest(population, k=5)
    population = toolbox.select(offspring, k=len(population) - 5) + elite

top_ind = tools.selBest(population, k=1)[0]
top_weights = np.array(top_ind[:2]) / np.sum(top_ind[:2])
w1, w2 = top_weights
threshold = top_ind[2]
fp_fn = evaluate([w1, w2, threshold])[0]

print("\n  Najlepšie nájdené váhy pre 2-modelový ensemble:")
print(f"yolov11    = {w1:.4f}")
print(f"convnexts  = {w2:.4f}")
print(f"threshold  = {threshold:.3f}")
print(f"fp + fn    = {fp_fn}")
```

[ ]:
```
# 1 model

df = pd.read_csv("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv")

def compute_fp_fn(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return fp + fn

def evaluate(individual):
    threshold = individual[0]
    df["convnext_prediction"] = (df["prediction_ConvNexts"] >= threshold).
 ↪astype(int)
    score = compute_fp_fn(df["true_label"], df["convnext_prediction"])
    return score,

creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

toolbox = base.Toolbox()
toolbox.register("individual", lambda: creator.Individual([
    random.uniform(0.1, 0.9)
]))
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

```
toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.05, indpb=0.5)
toolbox.register("select", tools.selTournament, tournsize=3)

population = toolbox.population(n=100)
NGEN = 50
CXPB = 0.7
MUTPB = 0.2

for gen in range(NGEN):
    offspring = algorithms.varAnd(population, toolbox, cxpb=CXPB, mutpb=MUTPB)
    fits = toolbox.map(toolbox.evaluate, offspring)
    for fit, ind in zip(fits, offspring):
        ind.fitness.values = fit
    elite = tools.selBest(population, k=3)
    population = toolbox.select(offspring, k=len(population) - 3) + elite

top_ind = tools.selBest(population, k=1)[0]
threshold = top_ind[0]
df["convnext_prediction"] = (df["prediction_ConvNexts"] >= threshold).
 ↪astype(int)
fp_fn = compute_fp_fn(df["true_label"], df["convnext_prediction"])

print("\n  Najlepšie nájdený threshold len pre ConvNext-Small:")
print(f"threshold  = {threshold:.4f}")
print(f"fp + fn    = {fp_fn}")
```

1.1.6 Vyhodnotenie výsledného kombinovaného modelu

[18]:
```
def compute_overlap(pos_scores, neg_scores, bins=20):
    pos_hist, bin_edges = np.histogram(pos_scores, bins=bins, range=(0, 1),␣
 ↪density=True)
    neg_hist, _ = np.histogram(neg_scores, bins=bins, range=(0, 1),␣
 ↪density=True)
    overlap = np.sum(np.minimum(pos_hist, neg_hist)) * (bin_edges[1] -␣
 ↪bin_edges[0])
    return overlap * 100

def evaluate_and_plot_plotly(df, nazov, threshold=0.5):
    y_true, y_scores = df['true_label'].values, df['confidence'].values

    precision_vals, recall_vals, thresholds = precision_recall_curve(y_true,␣
 ↪y_scores)
    f1_scores = 2 * (precision_vals * recall_vals) / (precision_vals +␣
 ↪recall_vals + 1e-6)
```

```
    max_f1_idx = np.argmax(f1_scores)
    best_f1_precision = precision_vals[max_f1_idx]
    best_f1_recall = recall_vals[max_f1_idx]

    max_precision_idx = np.argmax(precision_vals)
    best_prec_precision = precision_vals[max_precision_idx]
    best_prec_recall = recall_vals[max_precision_idx]

    y_pred = (y_scores >= threshold).astype(int)
    cm = confusion_matrix(y_true, y_pred, labels=[1, 0])
    TP, FN = cm[0]
    FP, TN = cm[1]

    precision = precision_score(y_true, y_pred, zero_division=0)
    recall = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
    accuracy = accuracy_score(y_true, y_pred)

    pos_scores = y_scores[y_true == 1]
    neg_scores = y_scores[y_true == 0]
    overlap_percent = compute_overlap(pos_scores, neg_scores)

    cm_text = np.array([
        [f"TP = {TP}", f"FN = {FN}"],
        [f"FP = {FP}", f"TN = {TN}"]
    ])

    cm_reordered = np.array([
        [TP, FN],
        [FP, TN]
    ])

    royal_blue = '#4169E1'
    yellow = '#FFD700'
    light_gray = '#F5F5F5'

    fig = sp.make_subplots(
        rows=1, cols=3,
        subplot_titles=(
            f"Chybová matica (Prah {threshold:.2f})",
            "PR Krivka",
            f"Distribúcia Pravdepodobností<br>Prekrytie: {overlap_percent:.2f}%"
        )
    )

    heatmap = go.Heatmap(
        z=cm_reordered,
```

```
        colorscale=[[0, light_gray], [1, royal_blue]],
        showscale=False,
        text=cm_text,
        texttemplate="%{text}"
    )

    fig.add_trace(heatmap, row=1, col=1)

    fig.add_annotation(
        x=-1.1, y=0.5, text="Skutočnosť",
        showarrow=False, font=dict(size=16), textangle=-90,
        xanchor="center", yanchor="middle", row=1, col=1
    )

    fig.add_annotation(
        x=0.5, y=2, text="Predikcia",
        showarrow=False, font=dict(size=16),
        xanchor="center", row=1, col=1
    )

    fig.add_annotation(
        x=0, y=1.7, text="S meteorom",
        showarrow=False, font=dict(size=14),
        xanchor="center", row=1, col=1
    )

    fig.add_annotation(
        x=1, y=1.7, text="Bez meteoru",
        showarrow=False, font=dict(size=14),
        xanchor="center", row=1, col=1
    )

    fig.add_annotation(
        x=-0.8, y=0, text="Bez meteoru",
        showarrow=False, font=dict(size=14), textangle=-90,
        yanchor="middle", xanchor="center", row=1, col=1
    )

    fig.add_annotation(
        x=-0.8, y=1, text="S meteorom",
        showarrow=False, font=dict(size=14), textangle=-90,
        yanchor="middle", xanchor="center", row=1, col=1
    )

    fig.add_trace(
        go.Scatter(
            x=recall_vals, y=precision_vals,
```

```
            mode='lines', line=dict(color=royal_blue, width=3),
            name='PR Krivka'
        ),
        row=1, col=2
    )

    fig.add_trace(
        go.Scatter(
            x=[best_f1_recall], y=[best_f1_precision],
            mode='markers+text', marker=dict(color=yellow, size=12,␣
↪symbol='circle'),
            text=["Max F1 Skóre"], textposition="bottom left",
            name='Max F1 Skóre'
        ),
        row=1, col=2
    )

    fig.add_trace(
        go.Scatter(
            x=[best_prec_recall],
            y=[best_prec_precision],
            mode='markers+text',
            marker=dict(color=yellow, size=12, symbol='diamond'),
            text=["Max Presnosť"],
            textposition="top right",
            name='Max Presnosť'
        ),
        row=1, col=2
    )

    threshold_idx = np.argmin(np.abs(thresholds - threshold))
    fig.add_trace(
        go.Scatter(
            x=[recall_vals[threshold_idx]],
            y=[precision_vals[threshold_idx]],
            mode='markers+text',
            marker=dict(color='black', size=12, symbol='circle'),
            text=[f"Prah {threshold:.2f}"], textposition="top left",
            name='Prah'
        ),
        row=1, col=2
    )

    fig.add_trace(
        go.Histogram(
            x=pos_scores, name='S meteorom',
            marker_color=royal_blue, opacity=0.75, nbinsx=20, showlegend=True
```

```
        ),
        row=1, col=3
    )

    fig.add_trace(
        go.Histogram(
            x=neg_scores, name='Bez meteoru',
            marker_color=yellow, opacity=0.75, nbinsx=20, showlegend=True
        ),
        row=1, col=3
    )

    fig.add_shape(
        type="line",
        x0=threshold, x1=threshold,
        y0=0, y1=1,
        line=dict(color="black", width=2, dash="dash"),
        xref='x3',
        yref='paper'
    )

    fig.update_layout(
        title=dict(
            text="Vyhodnotenie modelu: " + nazov,
            x=0.5, xanchor='center',
            font=dict(size=20)
        ),
        width=1600, height=450,
        barmode='overlay',
        plot_bgcolor=light_gray,
        paper_bgcolor='white',
        font=dict(size=13, color='black'),
        margin=dict(l=100, r=2, t=100, b=20),
        legend=dict(orientation="h", yanchor="bottom", y=1.05,␣
↪xanchor="center", x=0.5)
    )

    fig.update_xaxes(showgrid=False)
    fig.update_yaxes(showgrid=False)

    fig.update_xaxes(title_text="Návratnosť (Recall)", row=1, col=2)
    fig.update_yaxes(title_text="Presnosť (Precision)", row=1, col=2)
    fig.update_xaxes(title_text="Pravdepodobnosť meteoru", row=1, col=3)
    fig.update_yaxes(title_text="Počet obrázkov", row=1, col=3)

    fig.update_xaxes(zeroline=False, row=1, col=1, showticklabels=False)
    fig.update_yaxes(zeroline=False, row=1, col=1, showticklabels=False)
```

```
    fig.update_xaxes(zeroline=False, row=1, col=2, showticklabels=True)
    fig.update_yaxes(zeroline=False, row=1, col=2, showticklabels=True)

    fig.update_xaxes(zeroline=False, row=1, col=3, showticklabels=True)
    fig.update_yaxes(zeroline=False, row=1, col=3, showticklabels=True)

    print(f"\n  Výsledky pre prah {threshold:.2f}:")
    print(f"  TP (Správne meteory): {TP}")
    print(f"  TN (Správne nemeteory): {TN}")
    print(f"  FP (Falošné meteory): {FP}")
    print(f"  FN (Nezachytené meteory): {FN}")
    print(f"  Presnosť (Precision): {precision:.4f}")
    print(f"  Návratnosť (Recall): {recall:.4f}")
    print(f"  F1-Skóre: {f1:.4f}")
    print(f"  Presnosť klasifikácie (Accuracy): {accuracy:.4f}")
    print(f"  Prekrytie distribúcie (Overlap coefficient): {overlap_percent:.
 ↪2f}%")

    fig.show()

    print(f"\n  Max F1 bod: Návratnosť={best_f1_recall:.3f},␣
 ↪Presnosť={best_f1_precision:.3f}")
    print(f"  Max Presnosť bod: Návratnosť={best_prec_recall:.3f},␣
 ↪Presnosť={best_prec_precision:.3f}")
```

[19]:
```
df = pd.read_csv("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv")

weights = {
    'convnexts': 0.5424,
    'yolov11': 0.4576
}

df["ensemble_probability"] = (
    df["prediction_ConvNexts"] * weights["convnexts"] +
    df["prediction_yolov11"] * weights["yolov11"]
)

threshold = 0.441
df["ensemble_prediction"] = (df["ensemble_probability"] >= threshold).
 ↪astype(int)

print("  Váhy použité pre ensemble:")
for model, w in weights.items():
    print(f"{model}: {w:.2f}")
```

```
print("\n Správa klasifikácie (ensemble):")
print(classification_report(df["true_label"], df["ensemble_prediction"],␣
 ↪digits=4))

df_preds = pd.DataFrame({
    'true_label': df["true_label"],
    'predicted_label': df["ensemble_prediction"],
    'confidence': df["ensemble_probability"]
})

print(df_preds.head())

evaluate_and_plot_plotly(df_preds, "Finálny kombinovaný model pozostávajúci z␣
 ↪Yolov11 a ConvNeXt-Small modelov", threshold)
```

```
 Váhy použité pre ensemble:
convnexts: 0.54
yolov11: 0.46

 Správa klasifikácie (ensemble):
              precision    recall  f1-score   support

           0     0.9589    0.9352    0.9469       324
           1     0.9655    0.9783    0.9719       600

    accuracy                         0.9632       924
   macro avg     0.9622    0.9568    0.9594       924
weighted avg     0.9631    0.9632    0.9631       924

   true_label  predicted_label  confidence
0           0                0    0.069520
1           0                0    0.115189
2           0                0    0.150171
3           0                0    0.084437
4           0                0    0.114539

 Výsledky pre prah 0.44:
 TP (Správne meteory): 587
 TN (Správne nemeteory): 303
 FP (Falošné meteory): 21
 FN (Nezachytené meteory): 13
 Presnosť (Precision): 0.9655
 Návratnosť (Recall): 0.9783
 F1-Skóre: 0.9719
 Presnosť klasifikácie (Accuracy): 0.9632
 Prekrytie distribúcie (Overlap coefficient): 8.98%
```

```
Max F1 bod: Návratnosť=0.978, Presnosť=0.965
Max Presnosť bod: Návratnosť=0.167, Presnosť=1.000
```

Porovnanie zmien v klasifikáciách Yolov11 a kombinovaného modelu

[20]:
```python
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

# Načítanie dát
df = pd.read_csv("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv")

# Thresholdy
threshold_yolov11 = 0.275
threshold_ensemble = 0.441

# Natvrdo nastavené váhy
weights = {
    'convnexts': 0.5424,
    'yolov11': 0.4576
}

# Výpočet váženého priemeru pravdepodobností
df["ensemble_probability"] = (
    df["prediction_ConvNexts"] * weights["convnexts"] +
    df["prediction_yolov11"] * weights["yolov11"]
)

# Predikcie
df["yolov11_prediction"] = (df["prediction_yolov11"] >= threshold_yolov11).
 ↪astype(int)
df["ensemble_prediction"] = (df["ensemble_probability"] >= threshold_ensemble).
 ↪astype(int)
df["changed"] = df["yolov11_prediction"] != df["ensemble_prediction"]
df["change_label"] = df["changed"].map({True: "Zmenené predikcie", False:␣
 ↪"Nezmenené predikcie"})

# Vždy nesprávne detegované
df["always_wrong"] = (
    (df["true_label"] != df["yolov11_prediction"]) &
    (df["true_label"] != df["ensemble_prediction"])
)
df["final_label"] = df.apply(
    lambda row: "Falošné predikcie" if row["always_wrong"]
    else row["change_label"], axis=1
```

```
)

# Plot s marginálnymi grafmi
fig = px.scatter(
    df,
    x="prediction_yolov11",
    y="ensemble_probability",
    color="final_label",
    hover_data=["image_name", "true_label", "yolov11_prediction",␣
 ↪"ensemble_prediction"],
    labels={
        "prediction_yolov11": "YOLOv11 predikcie",
        "ensemble_probability": "Predikcie kombinovaného modelu"
    },
    #title="Zmeny v klasifikáciách obrázkov po aplikovaní učenia súborom metód",
    color_discrete_map={
        "Zmenené predikcie": '#47D45A',
        "Nezmenené predikcie": "lightgray",
        "Falošné predikcie": "black"
    },
    marginal_x="histogram",     # môžeš zmeniť na "histogram", "box", "rug"
    marginal_y="histogram",
    height=900,
    width=1600
)

# Pridanie threshold čiar
fig.add_shape(
    type="line",
    x0=threshold_yolov11, x1=threshold_yolov11,
    y0=0, y1=1,
    line=dict(color="black", width=1, dash="dash"),
)
fig.add_shape(
    type="line",
    x0=0, x1=1,
    y0=threshold_ensemble, y1=threshold_ensemble,
    line=dict(color="black", width=1, dash="dash"),
)

# Pridanie neviditeľných scatter objektov pre legendu (prerušené čiary)
fig.add_trace(go.Scatter(
    x=[None], y=[None],
    mode='lines',
    line=dict(color='black', width=1, dash='dash'),
    name='Rozhodovacie prahy'
))
```

```
# Layout
fig.update_layout(
    xaxis=dict(
        range=[0, 1],
        title_font=dict(size=24),
        tickfont=dict(size=24)
    ),
    yaxis=dict(
        range=[0, 1],
        title_font=dict(size=24),
        tickfont=dict(size=24)
    ),
    plot_bgcolor='#F5F5F5',
    paper_bgcolor='#FFFFFF',
    legend=dict(orientation="h", yanchor="bottom", y=1, xanchor="center", x=0.
 ↪5, font=dict(size=22), title_text=""),
    #legend_title="Typ zmeny",
    font=dict(size=24, color='black'),
    #title_x=0.5
)

# Zväčšenie bodov
fig.update_traces(
    marker=dict(size=15),
    selector=dict(mode='markers')  # aplikuje len na scatter body
)

fig.show()
```

Analýza zmien v predikciách po aplikácii učenia súborom metód

[21]:
```
import pandas as pd
import plotly.graph_objects as go

# Farby
royal_blue = '#4169E1'
yellow = '#FFD700'
magenta = '#E34FDC'
green = '#47D45A'
orange = '#FF9900'

# Načítanie dát
df = pd.read_csv("/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv")
```

```
# Thresholdy a váhy
threshold_yolov11 = 0.275
threshold_ensemble = 0.441
weights = {'convnexts': 0.5424, 'yolov11': 0.4576}

# Výpočty
df["ensemble_probability"] = (
    df["prediction_ConvNexts"] * weights["convnexts"] +
    df["prediction_yolov11"] * weights["yolov11"]
)
df["yolov11_prediction"] = (df["prediction_yolov11"] >= threshold_yolov11).
 ↪astype(int)
df["ensemble_prediction"] = (df["ensemble_probability"] >= threshold_ensemble).
 ↪astype(int)
df["changed"] = df["yolov11_prediction"] != df["ensemble_prediction"]

# Prípady, kde YOLOv11 bol správny a ensemble zlyhal
wrong_ensemble_mask = (
    (df["yolov11_prediction"] == df["true_label"]) &
    (df["ensemble_prediction"] != df["true_label"])
)

# Filtrovanie obrázkov na zobrazenie
df_changed = df[df["changed"]]
df_display = pd.concat([df_changed, df[wrong_ensemble_mask]]).
 ↪drop_duplicates(subset="image_name")

# Zistenie farby pre každý stĺpec kombinovaného modelu
ensemble_colors = [
    orange if wrong_ensemble_mask.loc[idx] else green
    for idx in df_display.index
]

# Vykreslenie
fig = go.Figure(data=[
    go.Bar(name='YOLOv11', x=df_display["image_name"],␣
 ↪y=df_display["prediction_yolov11"], marker_color=royal_blue),
    go.Bar(name='ConvNext-S', x=df_display["image_name"],␣
 ↪y=df_display["prediction_ConvNexts"], marker_color=yellow),
    go.Bar(name='Kombinovaná pravdepodobnosť', x=df_display["image_name"],␣
 ↪y=df_display["ensemble_probability"], marker_color=ensemble_colors),

    # Neviditeľné scatter objekty pre legendu
    go.Scatter(
        x=[None], y=[None], mode='lines',
        line=dict(color='black', width=2, dash='dash'),
```

```
        name=f'Rozhod. prah YOLOv11 ({threshold_yolov11})'
    ),
    go.Scatter(
        x=[None], y=[None], mode='lines',
        line=dict(color=magenta, width=2, dash='dash'),
        name=f'Rozhod. prah komb. modelu ({threshold_ensemble})'
    ),
    # Fiktívne pre farbu v legende
    go.Bar(x=[None], y=[None], name='Kombinovaná predikcia, správne␣
 ↪korigovaná', marker_color=green),
    go.Bar(x=[None], y=[None], name='Kombinovaná predikcia, nesprávne␣
 ↪korigovaná', marker_color=orange)
])

# Skutočné horizontálne čiary
fig.add_hline(y=threshold_ensemble, line_dash="dash", line_color=magenta)
fig.add_hline(y=threshold_yolov11, line_dash="dash", line_color="black")

# Layout
fig.update_layout(
    title="Analýza správne korigovaných a zhoršených predikcií po aplikovaní␣
 ↪učenia súborom metód",
    xaxis_title="Obrázky",
    yaxis_title="Predikované pravdepodobnosti",
    barmode='group',
    xaxis_tickangle=-60,
    height=800,
    width=1600,
    plot_bgcolor='#F5F5F5',
    paper_bgcolor='#FFFFFF',
    font=dict(size=12, color='black'),
    legend=dict(
        title_font=dict(size=20),
        font=dict(size=22)
    )
)

fig.update_xaxes(title_font=dict(size=26), tickfont=dict(size=14))
fig.update_yaxes(title_font=dict(size=26), tickfont=dict(size=22))


fig.show()
```

1.1.7 Analýza klasifikačných rozhodnutí kombinovaného modelu a ich interpretácia

ConvNext-Small

[22]:
```
import os
import torch
import torch.nn as nn
import timm
import numpy as np
from torchvision import transforms
from PIL import Image
from sklearn.manifold import TSNE
import plotly.express as px
import pandas as pd

import plotly.graph_objects as go
from scipy.spatial import ConvexHull
import hdbscan

# Cesty
test_images_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/vyhodnotenie/test/images"
test_labels_dir_classification = "/home/jovyan/data/lightning/LiviaMurankova/
 ↪new/namnozene_meteory/najlepsie_modely/vyhodnotenie/test/labels"
model_path_convnext = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ConvNext-Small/datasetv2_best_convnext_model_v65_epocha2.
 ↪pth"
csv_predictions_path = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv"

# Ensemble váhy a threshold
threshold_ensemble = 0.441
weights = {'convnexts': 0.5424, 'yolov11': 0.4576}

# Načítanie CSV predikcií
df_preds = pd.read_csv(csv_predictions_path)
df_preds["ensemble_probability"] = (
    df_preds["prediction_ConvNexts"] * weights["convnexts"] +
    df_preds["prediction_yolov11"] * weights["yolov11"]
)

# Torch device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Transformácie
transform = transforms.Compose([
    transforms.Resize((720, 720)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

```
# Načítanie modelu
convnext_model = timm.create_model("convnext_small", pretrained=False,␣
 ↪num_classes=1).to(device)
convnext_model.classifier = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(convnext_model.num_features, 1)
)
convnext_model.load_state_dict(torch.load(model_path_convnext,␣
 ↪map_location=device))
convnext_model.eval()

# Hook na získanie embeddings
embeddings_list = []
def get_features_hook(module, input, output):
    embeddings_list.append(output.detach().cpu().numpy())

convnext_model.stages[-1].register_forward_hook(get_features_hook)

# Extrakcia embeddings a labelov
image_paths = sorted([os.path.join(test_images_dir, fname) for fname in os.
 ↪listdir(test_images_dir) if fname.endswith(".jpg")])
data = []

for path in image_paths:
    filename = os.path.basename(path)
    label_path = os.path.join(test_labels_dir_classification, filename.
 ↪replace(".jpg", ".txt"))

    # Načítanie true labelu
    if os.path.exists(label_path):
        with open(label_path, "r") as f:
            content = f.read().strip()
        true_label = 1 if content == "0" else 0
    else:
        true_label = 0

    # Spracovanie obrázka
    image = Image.open(path).convert('RGB')
    input_tensor = transform(image).unsqueeze(0).to(device)

    with torch.no_grad():
        _ = convnext_model(input_tensor)

    embedding = embeddings_list[-1].squeeze()
    if embedding.ndim == 3:
        embedding = embedding.mean(axis=(1, 2))
```

```
    data.append({
        "filename": filename,
        "true_label": true_label,
        "embedding": embedding
    })

# Vytvorenie DataFrame
df_embed = pd.DataFrame(data)
df_embed["filename_base"] = df_embed["filename"].apply(lambda x: os.path.
 ↪splitext(x)[0])

# Pripojenie ensemble predikcií
df_preds["filename_base"] = df_preds["image_name"].apply(lambda x: os.path.
 ↪splitext(x)[0])
df_merged = pd.merge(df_embed, df_preds, on="filename_base", how="inner")

# Uisti sa, že 'true_label' je zo správneho zdroja
df_merged["true_label"] = df_merged["true_label_x"]


# Výpočet typu chyby podľa ensemble predikcie
def get_error_type(row):
    pred = 1 if row["ensemble_probability"] >= threshold_ensemble else 0
    true = row["true_label"]
    if pred == 1 and true == 1:
        return "TP"
    elif pred == 0 and true == 0:
        return "TN"
    elif pred == 1 and true == 0:
        return "FP"
    else:
        return "FN"

df_merged["error_type"] = df_merged.apply(get_error_type, axis=1)

# t-SNE vizualizácia
embeddings_array = np.stack(df_merged["embedding"].values)
tsne = TSNE(n_components=2, random_state=42)
tsne_result = tsne.fit_transform(embeddings_array)
df_merged["tsne_x"] = tsne_result[:, 0]
df_merged["tsne_y"] = tsne_result[:, 1]

# Farebná mapa
color_map = {
    "TP": "#4169E1",  # royal_blue
    "TN": "#FFD700",  # yellow
```

```
    "FP": "#E34FDC",  # magenta
    "FN": "#47D45A",  # green
}

# 1. Vytvor základnú figúru bez bodov (obaly pôjdu ako prvé)
fig = go.Figure()

# Farby pre obaly zhlukov
# Spustenie HDBSCAN na t-SNE výstupoch
clusterer = hdbscan.HDBSCAN(min_cluster_size=15, min_samples=5)
df_merged["cluster"] = clusterer.fit_predict(df_merged[["tsne_x", "tsne_y"]])

# Farby obalov pre zhluky (voliteľne nastav vlastné alebo náhodné)
cluster_colors = px.colors.qualitative.Set3
used_clusters = sorted([cid for cid in df_merged["cluster"].unique() if cid !=␣
 ↪-1])

# 2. Najprv vykresli konvexné obaly
for i, cluster_id in enumerate(used_clusters):
    cluster_df = df_merged[df_merged["cluster"] == cluster_id]
    if len(cluster_df) < 3:
        continue  # nemá zmysel kresliť obal

    points = cluster_df[["tsne_x", "tsne_y"]].values
    hull = ConvexHull(points)
    hull_points = points[hull.vertices]
    hull_points = np.append(hull_points, [hull_points[0]], axis=0)

    fig.add_trace(go.Scatter(
        x=hull_points[:, 0],
        y=hull_points[:, 1],
        fill='toself',
        mode='lines',
        line=dict(color=cluster_colors[i % len(cluster_colors)], width=2),
        fillcolor=cluster_colors[i % len(cluster_colors)].replace("rgb",␣
 ↪"rgba").replace(")", ",0.1)"),
        name=f"Zhluk {cluster_id}",
        hoverinfo='skip',
        showlegend=False
    ))

# 3. Potom pridaj body (farebne podľa chyby)
for err_type, color in color_map.items():
    subset = df_merged[df_merged["error_type"] == err_type]
    fig.add_trace(go.Scatter(
        x=subset["tsne_x"],
        y=subset["tsne_y"],
```

```
        mode="markers",
        name=err_type,
        marker=dict(size=15, color=color, opacity=0.8),
        hovertemplate="<b>%{text}</b><br>Label: %{customdata[0]}<br>Ensemble:␣
 ↪%{customdata[1]:.3f}<extra></extra>",
        text=subset["filename"],
        customdata=np.stack([subset["true_label"],␣
 ↪subset["ensemble_probability"]], axis=-1)
    ))

# 4. Nakoniec pridaj textové labely zhlukov
for cluster_id in used_clusters:
    cluster_df = df_merged[df_merged["cluster"] == cluster_id]
    center_x = cluster_df["tsne_x"].mean()
    center_y = cluster_df["tsne_y"].mean()
    fig.add_trace(go.Scatter(
        x=[center_x],
        y=[center_y],
        mode='text',
        text=[f"Zhluk {cluster_id}"],
        textposition="top center",
        showlegend=False,
        hoverinfo='skip',
        textfont=dict(size=26, color='black', family='Arial')
    ))

# 5. Layout a vzhľad
fig.update_layout(
    title="t-SNE vizualizácia ConvNeXt vektorov<br><sup>Farebne podľa predikcií␣
 ↪kombinovaného modelu, obalené podľa HDBSCAN zhlukov</sup>",
    plot_bgcolor='#F5F5F5',
    paper_bgcolor='#FFFFFF',
    font=dict(size=18, color='black'),
    legend=dict(
        title="Typ chyby",
        title_font=dict(size=24),
        font=dict(size=24),
        orientation="h",
        yanchor="bottom",
        y=1.02,
        xanchor="center",
        x=0.5
    ),
    height=1000,
    width=1600,
    margin=dict(t=170)
)
```

```
fig.show()
```

```
/opt/conda/lib/python3.12/site-packages/sklearn/utils/deprecation.py:151:
FutureWarning:

'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed
in 1.8.

/opt/conda/lib/python3.12/site-packages/sklearn/utils/deprecation.py:151:
FutureWarning:

'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed
in 1.8.
```

Yolov11

[24]:
```
import os
import torch
import numpy as np
import pandas as pd
import ultralytics

from PIL import Image
from sklearn.manifold import TSNE
import plotly.express as px
from torchvision import transforms
from ultralytics import YOLO

# Cesty
model_path_yolov11 = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/najlepsie_modely/yolov11_vacsie_obrazky.
 ↪pt"
test_images_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/vyhodnotenie/test/images"
test_labels_dir_detection = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/vyhodnotenie/test_yolo/labels"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Načítanie YOLOv11 modelu
yolo_model_11 = YOLO(model_path_yolov11).to(device)
yolo_model_11.eval()

# Transformácia na veľkosť 1408 × 1408 (tak bola sieť trénovaná)
yolo_transform = transforms.Compose([
    transforms.Resize((1408, 1408)),
```

```
    transforms.ToTensor(),
])

# Funkcia na extrakciu embeddings (pred detekčnou hlavou)
def extract_yolo_features(image_path):
    image = Image.open(image_path).convert("RGB")
    image_tensor = yolo_transform(image).unsqueeze(0).to(device)

    x = image_tensor
    outputs = []  # pre Concat vrstvy

    with torch.no_grad():
        for idx, layer in enumerate(yolo_model_11.model.model):
            if isinstance(layer, torch.nn.modules.container.Sequential) and␣
 ↪any(isinstance(mod, YOLO) for mod in layer):
                break  # bezpečnostná poistka

            if isinstance(layer, ultralytics.nn.modules.conv.Concat):
                # zober vstupy podľa požadovaných indexov
                x = layer([outputs[i] for i in layer.f])
            else:
                x = layer(x)

            outputs.append(x)

            if idx == 22:  # posledná vrstva pred 'Detect'
                break

        pooled = torch.mean(x, dim=(2, 3)).squeeze().cpu().numpy()  # GAP

    return pooled

# Funkcia na výpočet confidence YOLO detekcií
def predict_yolo_confidence(image_path, threshold=0.275):
    with torch.no_grad():
        results = yolo_model_11(image_path, verbose=False)
        detections = results[0].boxes
    confidences = [det[4] for det in detections.data.cpu().numpy() if det[4] >␣
 ↪threshold]
    return max(confidences) if confidences else 0.0

# Spracovanie testovacích obrázkov
image_paths = sorted([os.path.join(test_images_dir, f) for f in os.
 ↪listdir(test_images_dir) if f.endswith(".jpg")])
data = []

for path in image_paths:
```

```
    filename = os.path.basename(path)
    label_path = os.path.join(test_labels_dir_detection, filename.replace(".
 ↪jpg", ".txt"))

    # Načítanie true labelu: ak prvý riadok začína "0", je tam meteor
    if os.path.exists(label_path):
        with open(label_path, "r") as f:
            lines = f.readlines()
        true_label = 1 if lines and lines[0].strip().startswith("0") else 0
    else:
        true_label = 0

    # Predikcia + embedding
    pred_prob = predict_yolo_confidence(path)
    embedding = extract_yolo_features(path)

    predicted_label = 1 if pred_prob >= 0.275 else 0
    correct = int(predicted_label == true_label)

    if predicted_label == 1 and true_label == 1:
        error_type = "TP"
    elif predicted_label == 0 and true_label == 0:
        error_type = "TN"
    elif predicted_label == 1 and true_label == 0:
        error_type = "FP"
    else:
        error_type = "FN"

    data.append({
        "filename": filename,
        "true_label": true_label,
        "predicted_prob": pred_prob,
        "predicted_label": predicted_label,
        "correct": correct,
        "embedding": embedding,
        "error_type": error_type
    })

# t-SNE zníženie rozmerov
embeddings = np.array([item["embedding"] for item in data])
tsne = TSNE(n_components=2, random_state=42)
tsne_results = tsne.fit_transform(embeddings)

for i, coords in enumerate(tsne_results):
    data[i]["tsne_x"] = coords[0]
    data[i]["tsne_y"] = coords[1]
```

```
# Vizuál
df = pd.DataFrame(data)
df["error_type"] = df["error_type"].astype(str)

# === CSV s ensemble predikciami ===
csv_predictions_path = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/
 ↪all_predictions_yolov11_yolov12_convnextv65_efficientNetB4.csv"
df_preds = pd.read_csv(csv_predictions_path)

# Výpočet ensemble pravdepodobnosti
threshold_ensemble = 0.441
weights = {'convnexts': 0.5424, 'yolov11': 0.4576}
df_preds["ensemble_probability"] = (
    df_preds["prediction_ConvNexts"] * weights["convnexts"] +
    df_preds["prediction_yolov11"] * weights["yolov11"]
)

# Pridanie filename_base pre join
df_preds["filename_base"] = df_preds["image_name"].str.replace(".jpg", "",␣
 ↪regex=False)

# Príprava df z YOLO výpočtu
df = pd.DataFrame(data)
df["filename_base"] = df["filename"].str.replace(".jpg", "", regex=False)

# Merge YOLO embeddings + ensemble predikcie
df_merged = pd.merge(df, df_preds, on="filename_base", how="inner")

# Zabezpeč, že true_label je z YOLO dát
df_merged["true_label"] = df_merged["true_label_x"]

# Výpočet typu chyby podľa ensemble predikcie
def get_error_type(row):
    pred = 1 if row["ensemble_probability"] >= threshold_ensemble else 0
    true = row["true_label"]
    if pred == 1 and true == 1:
        return "TP"
    elif pred == 0 and true == 0:
        return "TN"
    elif pred == 1 and true == 0:
        return "FP"
    else:
        return "FN"

df_merged["error_type_ensemble"] = df_merged.apply(get_error_type, axis=1)
```

```
# Výpis počtu
counts = df_merged["error_type_ensemble"].value_counts().to_dict()
print("  Počet vzoriek podľa ensemble predikcie:")
print(f"  True Positive (TP):   {counts.get('TP', 0)}")
print(f"  True Negative (TN):   {counts.get('TN', 0)}")
print(f"  False Positive (FP):  {counts.get('FP', 0)}")
print(f"  False Negative (FN):  {counts.get('FN', 0)}")

# t-SNE vizualizácia
fig = px.scatter(
    df_merged,
    x="tsne_x",
    y="tsne_y",
    color="error_type_ensemble",
    hover_data=["filename", "true_label", "ensemble_probability"],
    title="t-SNE vizualizácia YOLOv11 embeddings podľa ENSEMBLE predikcie",
    color_discrete_map={
        "TP": "#4169E1",  # royal blue
        "TN": "#FFD700",  # yellow
        "FP": "#E34FDC",  # magenta
        "FN": "#47D45A",  # green
    }
)

fig.update_layout(
    plot_bgcolor='#F5F5F5',
    paper_bgcolor='#FFFFFF',
    font=dict(size=20, color='black'),
    legend=dict(
        title="Typ chyby (ensemble)",
        title_font=dict(size=20),
        font=dict(size=20)
    )
)
fig.update_traces(marker=dict(size=8, opacity=0.8))
fig.show()
```

```
 Počet vzoriek podľa ensemble predikcie:
 True Positive (TP):   587
 True Negative (TN):   303
 False Positive (FP):  21
 False Negative (FN):  13
```

[25]:
```
import plotly.graph_objects as go
from scipy.spatial import ConvexHull
import hdbscan
```

```
# Farebná mapa
color_map = {
    "TP": "#4169E1",  # royal_blue
    "TN": "#FFD700",  # yellow
    "FP": "#E34FDC",  # magenta
    "FN": "#47D45A",  # green
}

# 1. Vytvor základnú figúru bez bodov (obaly pôjdu ako prvé)
fig = go.Figure()

# Spusti HDBSCAN na t-SNE výstupy
tsne_features = df_merged[["tsne_x", "tsne_y"]].values
clusterer = hdbscan.HDBSCAN(min_cluster_size=15, min_samples=8)
cluster_labels = clusterer.fit_predict(tsne_features)

# Pridaj výsledky do DataFrame
df_merged["cluster"] = cluster_labels

# Farby pre obaly zhlukov
cluster_colors = px.colors.qualitative.Set3
used_clusters = sorted([cid for cid in df_merged["cluster"].unique() if cid !=␣
 ↪-1])

# 2. Najprv vykresli konvexné obaly
for i, cluster_id in enumerate(used_clusters):
    cluster_df = df_merged[df_merged["cluster"] == cluster_id]
    if len(cluster_df) < 3:
        continue  # nemá zmysel kresliť obal

    points = cluster_df[["tsne_x", "tsne_y"]].values
    hull = ConvexHull(points)
    hull_points = points[hull.vertices]
    hull_points = np.append(hull_points, [hull_points[0]], axis=0)

    fig.add_trace(go.Scatter(
        x=hull_points[:, 0],
        y=hull_points[:, 1],
        fill='toself',
        mode='lines',
        line=dict(color=cluster_colors[i % len(cluster_colors)], width=2),
        fillcolor=cluster_colors[i % len(cluster_colors)].replace("rgb",␣
 ↪"rgba").replace(")", ",0.1)"),
        name=f"Zhluk {cluster_id}",
        hoverinfo='skip',
        showlegend=False
    ))
```

```
# 3. Potom pridaj body (farebne podľa chyby)
for err_type, color in color_map.items():
    subset = df_merged[df_merged["error_type_ensemble"] == err_type]
    fig.add_trace(go.Scatter(
        x=subset["tsne_x"],
        y=subset["tsne_y"],
        mode="markers",
        name=err_type,
        marker=dict(size=15, color=color, opacity=0.8),
        hovertemplate="<b>%{text}</b><br>Label: %{customdata[0]}<br>Ensemble:␣
 ↪%{customdata[1]:.3f}<extra></extra>",
        text=subset["filename"],
        customdata=np.stack([subset["true_label"],␣
 ↪subset["ensemble_probability"]], axis=-1)
    ))

# 4. Nakoniec pridaj textové labely zhlukov
for cluster_id in used_clusters:
    cluster_df = df_merged[df_merged["cluster"] == cluster_id]
    center_x = cluster_df["tsne_x"].mean()
    center_y = cluster_df["tsne_y"].mean()
    fig.add_trace(go.Scatter(
        x=[center_x],
        y=[center_y],
        mode='text',
        text=[f"Zhluk {cluster_id}"],
        textposition="top center",
        showlegend=False,
        hoverinfo='skip',
        textfont=dict(size=26, color='black', family='Arial')
    ))

# 5. Layout a vzhľad
fig.update_layout(
    title="t-SNE vizualizácia yolov11 vektorov<br><sup>Farebne podľa predikcií␣
 ↪kombinovaného modelu, obalené podľa HDBSCAN zhlukov</sup>",
    plot_bgcolor='#F5F5F5',
    paper_bgcolor='#FFFFFF',
    font=dict(size=18, color='black'),
    legend=dict(
        title="Typ chyby",
        title_font=dict(size=24),
        font=dict(size=24),
        orientation="h",
        yanchor="bottom",
        y=1.02,
```

```
        xanchor="center",
        x=0.5
    ),
    height=1000,
    width=1600,
    margin=dict(t=170)
)

fig.show()
```

```
/opt/conda/lib/python3.12/site-packages/sklearn/utils/deprecation.py:151:
FutureWarning:

'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed
in 1.8.

/opt/conda/lib/python3.12/site-packages/sklearn/utils/deprecation.py:151:
FutureWarning:

'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed
in 1.8.
```

1.1.8 Gradcam vizualizácie najviac aktivovaných častí obrazu

[ ]:
```
import os
import torch
import torch.nn.functional as F
import timm
import cv2
import numpy as np
from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt

# Cesty
test_images_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/najlepsie_modely/vyhodnotenie/test/images"
model_path = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪ConvNext-Small/datasetv2_best_convnext_model_v65_epocha2.pth"
output_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪ensemble_learning/gradcam_vizualizacie"
os.makedirs(output_dir, exist_ok=True)

# Transformácie
transform = transforms.Compose([
    transforms.Resize((720, 720)),
```

```
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Načítanie modelu
model = timm.create_model("convnext_small", pretrained=False, num_classes=1)
model.classifier = torch.nn.Sequential(
    torch.nn.Dropout(0.5),
    torch.nn.Linear(model.num_features, 1)
)
model.load_state_dict(torch.load(model_path, map_location=device))
model.to(device).eval()

# Hooky pre aktivácie a gradienty
activations = []
gradients = []

def forward_hook(module, input, output):
    activations.append(output)

def backward_hook(module, grad_input, grad_output):
    gradients.append(grad_output[0])

target_layer = model.stages[-1]
target_layer.register_forward_hook(forward_hook)
target_layer.register_full_backward_hook(backward_hook)

# Grad-CAM výpočet
def generate_gradcam(image_tensor):
    activations.clear()
    gradients.clear()

    image_tensor = image_tensor.unsqueeze(0).to(device).requires_grad_()
    output = model(image_tensor)

    model.zero_grad()
    output.backward(torch.ones_like(output))

    activation = activations[0].squeeze(0)  # shape: (C, H, W)
    gradient = gradients[0].squeeze(0)      # shape: (C, H, W)

    weights = gradient.mean(dim=(1, 2))     # priemer cez H a W
    cam = torch.zeros_like(activation[0])
    for i, w in enumerate(weights):
```

```
        cam += w * activation[i]

    cam = F.relu(cam)
    cam -= cam.min()
    cam /= (cam.max() + 1e-5)
    return cam.detach().cpu().numpy()

# Vykreslenie a uloženie výsledkov
def save_gradcam_visualization(image_path, output_path):
    image = Image.open(image_path).convert('RGB')
    input_tensor = transform(image)

    cam = generate_gradcam(input_tensor)

    cam_resized = cv2.resize(cam, image.size)
    heatmap = cv2.applyColorMap(np.uint8(255 * cam_resized), cv2.COLORMAP_JET)
    image_np = np.array(image)
    overlay = 0.5 * image_np + 0.5 * heatmap
    overlay = np.clip(overlay, 0, 255).astype(np.uint8)

    cv2.imwrite(output_path, overlay[:, :, ::-1])  # RGB -> BGR

# Spracovanie všetkých obrázkov
image_files = [f for f in os.listdir(test_images_dir) if f.endswith(".jpg")]
for fname in image_files:
    img_path = os.path.join(test_images_dir, fname)
    out_path = os.path.join(output_dir, fname)
    save_gradcam_visualization(img_path, out_path)

print(" Grad-CAM vizualizácie boli úspešne uložené.")
```

[ ]:
```
import os
import torch
import torch.nn.functional as F
import timm
import cv2
import numpy as np
from torchvision import transforms
from PIL import Image

#  Cesty
clustered_base_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/tSNE"
output_base_dir = "/home/jovyan/data/lightning/LiviaMurankova/new/
 ↪namnozene_meteory/ensemble_learning/gradcam_vizualizacie"
model_path = "/home/jovyan/data/lightning/LiviaMurankova/new/namnozene_meteory/
 ↪ConvNext-Small/datasetv2_best_convnext_model_v65_epocha2.pth"
```

```python
# 🔹 Transformácie
transform = transforms.Compose([
    transforms.Resize((720, 720)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# 🔹 Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 🔹 Grad-CAM pre každý zhluk
cluster_dirs = sorted([d for d in os.listdir(clustered_base_dir) if os.path.
 ↪isdir(os.path.join(clustered_base_dir, d))])

for cluster_name in cluster_dirs:
    print(f"\n🔍 Spracovávam {cluster_name}...")
    input_dir = os.path.join(clustered_base_dir, cluster_name)
    output_dir = os.path.join(output_base_dir, cluster_name)
    os.makedirs(output_dir, exist_ok=True)

    # 🔹 Načítanie modelu pre každý zhluk (kvôli uvoľneniu GPU pamäte)
    torch.cuda.empty_cache()
    model = timm.create_model("convnext_small", pretrained=False, num_classes=1)
    model.classifier = torch.nn.Sequential(
        torch.nn.Dropout(0.5),
        torch.nn.Linear(model.num_features, 1)
    )
    model.load_state_dict(torch.load(model_path, map_location=device))
    model.to(device).eval()

    activations = []
    gradients = []

    def forward_hook(module, input, output):
        activations.append(output)

    def backward_hook(module, grad_input, grad_output):
        gradients.append(grad_output[0])

    target_layer = model.stages[-1]
    target_layer.register_forward_hook(forward_hook)
    target_layer.register_full_backward_hook(backward_hook)

    def generate_gradcam(image_tensor):
        activations.clear()
        gradients.clear()
```

```
        image_tensor = image_tensor.unsqueeze(0).to(device).requires_grad_()
        output = model(image_tensor)
        model.zero_grad()
        output.backward(torch.ones_like(output))

        activation = activations[0].squeeze(0)
        gradient = gradients[0].squeeze(0)
        weights = gradient.mean(dim=(1, 2))
        cam = torch.zeros_like(activation[0])
        for i, w in enumerate(weights):
            cam += w * activation[i]

        cam = F.relu(cam)
        cam -= cam.min()
        cam /= (cam.max() + 1e-5)
        return cam.detach().cpu().numpy()

    def save_gradcam_visualization(image_path, output_path):
        image = Image.open(image_path).convert('RGB')
        input_tensor = transform(image)
        cam = generate_gradcam(input_tensor)
        cam_resized = cv2.resize(cam, image.size)
        heatmap = cv2.applyColorMap(np.uint8(255 * cam_resized), cv2.
↪COLORMAP_JET)
        image_np = np.array(image)
        overlay = 0.5 * image_np + 0.5 * heatmap
        overlay = np.clip(overlay, 0, 255).astype(np.uint8)
        cv2.imwrite(output_path, overlay[:, :, ::-1])  # RGB -> BGR

    image_files = [f for f in os.listdir(input_dir) if f.endswith(".jpg")]
    for fname in image_files:
        img_path = os.path.join(input_dir, fname)
        out_path = os.path.join(output_dir, fname)
        save_gradcam_visualization(img_path, out_path)

    print(f"  Hotovo: {cluster_name} ({len(image_files)} obrázkov)")

    #  Uvoľnenie GPU pamäte
    del model
    torch.cuda.empty_cache()
```

1.1.9 Nasadenie

Dockerfile

[ ]:
```
FROM python:3.12-slim

# Inštalácia závislostí
RUN apt-get update && apt-get install -y \
    build-essential \
    libgl1-mesa-glx \
    libglib2.0-0 \
    libsm6 \
    libxrender1 \
    libxext6 \
    && rm -rf /var/lib/apt/lists/*

# Aktualizácia pip a inštalácia rovnakých verzií knižníc ako v Datalabe
RUN pip install --upgrade pip

# Inštalácia CUDA-kompatibilných torch balíčkov z oficiálneho indexu
RUN pip install torch==2.6.0 torchvision==0.21.0

# Ostatné knižnice
RUN pip install timm==1.0.15 ultralytics==8.3.102 pillow==11.1.0 numpy==1.26.4

# Pracovný adresár
WORKDIR /app

# Skopírovanie kódu
COPY . /app/

# Spustenie skriptu
ENTRYPOINT ["python", "process_images.py"]
```

process_images.py

[ ]:
```
import os
import argparse
import torch
import timm
import torch.nn as nn
from PIL import Image
from torchvision import transforms
from ultralytics import YOLO
from datetime import datetime

# === Argumenty ===
parser = argparse.ArgumentParser()
parser.add_argument("--original_input_dir", required=True, help="Lokálna cesta␣
 ↪k obrázkom")
args = parser.parse_args()
```

```
# === Dynamická cesta v kontajneri ===
original_input_dir = args.original_input_dir
subfolder = os.path.basename(original_input_dir)
image_root_dir = os.path.join("/app/images", subfolder)

# === Fixné cesty k modelom a výsledkom ===
convnext_path = "/app/models/convnext_model_legacy.pth"
yolov11_path = "/app/models/yolov11_model.pt"
output_dir = "/app/results"

device = torch.device("cpu")

# === Ensemble nastavenia ===
weights = {'convnexts': 0.5424, 'yolov11': 0.4576}
threshold = 0.441
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_path = os.path.join(output_dir, f"vysledky_{timestamp}.txt")

# === Transformácia ===
transform = transforms.Compose([
    transforms.Resize((720, 720)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

# === Načítanie modelov ===
convnext_model = timm.create_model("convnext_small", pretrained=False,␣
 ↪num_classes=1)
convnext_model.classifier = nn.Sequential(nn.Dropout(0.5), nn.
 ↪Linear(convnext_model.num_features, 1))
with open(convnext_path, 'rb') as f:
    convnext_model.load_state_dict(torch.load(f, map_location=device))
convnext_model.eval()

yolo_model = YOLO(yolov11_path).to(device)
yolo_model.eval()

# === Predikcie ===
def predict_convnext(image_path):
    image = Image.open(image_path).convert('RGB')
    image_tensor = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        return torch.sigmoid(convnext_model(image_tensor)).item()

def predict_yolo(image_path, threshold=0.0):
    with torch.no_grad():
```

```
        results = yolo_model(image_path, verbose=False)
        detections = results[0].boxes
    confidences = [det[4] for det in detections.data.cpu().numpy() if det[4] >␣
 ↪threshold]
    return max(confidences) if confidences else 0.0

# === Načítanie obrázkov ===
image_files = []
for root, _, files in os.walk(image_root_dir):
    for fname in files:
        if fname.lower().endswith(".jpg"):
            image_files.append(os.path.join(root, fname))

# === Vyhodnotenie a uloženie výsledkov ===
results = []
for img_path in sorted(image_files):
    pred_conv = predict_convnext(img_path)
    pred_yolo = predict_yolo(img_path)

    ensemble_prob = pred_conv * weights['convnexts'] + pred_yolo *␣
 ↪weights['yolov11']
    label = "Met" if ensemble_prob >= threshold else "Nemet"

    rel_path = os.path.relpath(img_path, image_root_dir).replace("\\", "/")
    original_path = f"{original_input_dir}/{rel_path}"

    results.append(f"{original_path} {ensemble_prob:.2f} {label}\n")

with open(output_path, "w") as f:
    f.writelines(results)

print(f"Výsledky sú uložené do {output_path}")
```

Príkazy na spustenie klasifikácií/predikcií finálneho modelu z terminálu

[ ]:
```
cd C:\Users\ago\Desktop\TUKE_AMOS

docker build -t meteordetection .

docker run --rm -v "C:/Users/ago/Desktop/TUKE_AMOS:/app" --entrypoint python␣
 ↪meteordetection process_images.py --original_input_dir "C:/Users/ago/Desktop/
 ↪TUKE_AMOS/images/AMOS_DATA_20250408"
```

[ ]:
```

```